A Simple, Optimal and Efficient Algorithm for Online Exp-Concave Optimization

Yi-Han Wang WANGYH@LAMDA.NJU.EDU.CN
Peng Zhao* ZHAOP@LAMDA.NJU.EDU.CN
Zhi-Hua Zhou ZHOUZH@LAMDA.NJU.EDU.CN
State Key Laboratory for Novel Software Technology, Nanjing University, China
School of Artificial Intelligence, Nanjing University, China

Editors: Steve Hanneke and Tor Lattimore

Abstract

Online eXp-concave Optimization (OXO) is a fundamental problem in online learning, where the goal is to minimize regret when loss functions are exponentially concave. The standard algorithm, Online Newton Step (ONS), guarantees an optimal $O(d \log T)$ regret, where d is the dimension and T is the time horizon. Despite its simplicity, ONS may face a computational bottleneck due to the *Mahalanobis projection* at each round. This step costs $\widetilde{O}(d^\omega)$ arithmetic operations for bounded domains, even for simple domains such as the unit ball, where $\omega \in (2, 3]$ is the matrix-multiplication exponent. As a result, the total runtime can reach $\widetilde{O}(d^\omega T)$, particularly when iterates frequently oscillate near the domain boundary. This paper proposes a simple variant of ONS, called LightONS, which reduces the total runtime to $O(d^2 T + d^\omega \sqrt{T \log T})$ while preserving the optimal regret. Deploying LightONS with the online-to-batch conversion implies a method for stochastic exp-concave optimization with runtime $\widetilde{O}(d^3/\varepsilon)$, thereby answering an open problem posed by Koren [2013]. The design leverages domain-conversion techniques from parameter-free online learning and defers expensive Mahalanobis projections until necessary, thereby preserving the elegant structure of ONS and enabling LightONS to act as an efficient plug-in replacement in broader scenarios, including gradient-norm adaptivity, parametric stochastic bandits, and memory-efficient OXO.

1. Introduction

Online Convex Optimization (OCO) provides a versatile framework for online learning, with deep connections to stochastic optimization, game theory, and information theory [Cesa-Bianchi and Lugosi, 2006; Hazan, 2016]. Online eXp-concave Optimization (OXO) is an important instance, where loss functions are exponentially concave (exp-concave), i.e., $\exp(-\alpha f(\cdot))$ is concave for some $\alpha > 0$ with f denoting the loss function. Exp-concavity naturally arises in many machine learning applications such as linear/logistic regression [Foster, 1991; Vovk, 1997; Foster et al., 2018], portfolio selection [Cover, 1991], linear-quadratic regulator control [Foster and Simchowitz, 2020], and so on. From a theoretical perspective, it introduces rich structures beyond convexity, allowing algorithms to exploit the local geometry of the loss landscape, often through local norms induced along the optimization trajectory. Such structures yield sharper statistical guarantees: Exp-concave losses admit a minimax-optimal regret of $O(d \log T)$ [Ordentlich and Cover, 1998], an exponential improvement over the $\Omega(\sqrt{T})$ lower bound for convex losses [Abernethy et al., 2008].

Online Newton Step (ONS) [Hazan et al., 2007] is the de facto standard for OXO, achieving an optimal regret bound of $O(d \log T)$ with constant time and space per round. ONS exhibits

* Corresponding Author.

© 2026 Y.-H. Wang, P. Zhao & Z.-H. Zhou.

remarkable simplicity, which has driven many advances in diverse optimization settings [Orabona et al., 2012; Luo et al., 2016; Cutkosky and Orabona, 2018] and in machine learning applications even beyond regret minimization, such as generalized linear bandits [Zhang et al., 2016; Jun et al., 2017; Zhang et al., 2025]. The important insight of ONS is to leverage exp-concavity by maintaining a *Hessian-related matrix* that captures the local geometry of the optimization trajectory, which is crucial for a balance between statistical optimality and computational practicality. Indeed, all known optimal OXO algorithms that avoid ONS-like Hessian maintenance, such as Exponential Weight Online Optimization (EWOO) [Hazan et al., 2007], incur prohibitive time complexity $O(T^{24})$ due to integrating over the domain [Bubeck et al., 2018].

However, the time-varying Hessian-related matrix in ONS necessitates a *Mahalanobis projection* at each round to ensure feasibility, which introduces the computational bottleneck. Specifically, the Mahalanobis projection solves the quadratic program $\Pi_{\mathcal{X}}^{M}[\mathbf{y}] = \arg\min_{\mathbf{x}\in\mathcal{X}}(\mathbf{x}-\mathbf{y})^{\mathsf{T}}M(\mathbf{x}-\mathbf{y})$ for some positive-definite and symmetric matrix M and compact convex domain $\mathcal{X} \subseteq \mathbb{R}^d$. When the domain $\mathcal{X}$ is bounded, best-known implementations require $\widetilde{O}(d^\omega)$ arithmetic operations due to matrix factorizations such as matrix square root [Golub and Van Loan, 2013], where $\omega \in (2, 3]$ is the matrix-multiplication exponent. [1] Even for simple domains such as the unit ball and the probability simplex, Mahalanobis projection requires $\widetilde{O}(d^\omega)$ time (see Appendix B.3) and $\widetilde{O}(d^{\omega+0.5})$ time (via interior-point methods [Nesterov and Nemirovskii, 1994]), respectively. Thus, the total runtime of ONS can reach $\widetilde{O}(d^\omega T)$, particularly when iterates frequently oscillate near the domain boundary. Although Alman et al. [2025] establish $\omega < 2.3714$ theoretically, linear algebra libraries typically operate with $\omega = 3$, yielding $\widetilde{O}(d^3T)$ time in practice. In contrast, for convex and strongly convex online optimization, Online Gradient Descent (OGD) [Zinkevich, 2003] requires only Euclidean projections and achieves minimax-optimal regret with a runtime of $O(dT)$ for these simple domains.

Similar computational challenges arise in Stochastic eXp-concave Optimization (SXO). As highlighted by a COLT'13 open problem of Koren [2013], ONS remains the default algorithm for SXO when equipped with the online-to-batch conversion. The optimal regret $O(d \log T)$ translates into an optimal sample complexity $T = \widetilde{O}(d/\varepsilon)$ for an excess risk of ε, where $\widetilde{O}(\cdot)$ hides poly-logarithmic factors in d/ε. Consequently, solving SXO with ONS incurs a total runtime of $\widetilde{O}(d^{\omega+1}/\varepsilon)$, which evaluates to $\widetilde{O}(d^4/\varepsilon)$ in practice. The open problem asks for an SXO algorithm with runtime below $\widetilde{O}(d^4/\varepsilon)$, i.e., one that achieves both statistical optimality and computational efficiency.

Related work. The pursuit of computationally efficient OXO algorithms can be divided into two main research lines. *(i)* The first, which we focus on in this paper, aims to minimize runtime while preserving optimal regret. The most closely related work is by Mhammedi and Gatmiry [2023], who proposed OQNS (Online Quasi-Newton Steps), achieving the optimal regret $O(d \log T)$ with runtime $O(d^2T \log T + d^\omega\sqrt{T \log T})$. They employ a log-barrier to eliminate Mahalanobis projections, transferring the computational burden to Hessian-inverse evaluations under log-barrier regularization, which is similar to interior-point methods [Hazan and Megiddo, 2023]. However, their method departs from canonical algorithmic frameworks such as online mirror descent (OMD) and Follow-the-Regularized-Leader (FTRL) [Orabona, 2019], making it difficult to accommodate diverse local norms and thereby limiting its applicability beyond standard OXO. A concrete comparison between our method and OQNS is deferred to Section 5. *(ii)* The second line aims to reduce regret within a time or space budget, for example via matrix sketching techniques [Luo et al., 2016]. These

1. The factorization underlying Mahalanobis projections is related to eigendecomposition, equivalent to finding roots of a d-degree polynomial, which is not exactly solvable by finitely many arithmetic operations when $d \geq 5$.

Table 1: Algorithmic upper bounds on regret and total runtime of OXO algorithms with respect to d and T over a simple domain of the unit ball. "ONS-Like" indicates whether the method can be integrated into other settings where ONS serves as the backbone, including gradient-norm adaptivity, parametric stochastic bandits, and memory-efficient OXO.

Algorithm	Regret	Total Runtime	ONS-Like
OGD [Zinkevich, 2003]	$\sqrt{T}$	dT	–
ONS [Hazan et al., 2007]	$d \log T$	$d^2T + d^\omega T \log T$	–
OQNS [Mhammedi and Gatmiry, 2023]	$d \log T$	$d^2T \log T + d^\omega \sqrt{T \log T}$	✗
LightONS (This Paper)	$d \log T$	$d^2T + d^\omega \sqrt{T \log T}$	✓

methods achieve linear-in-d runtime and working memory, albeit under additional assumptions on the loss functions and domains. This line also includes projection-free methods that trade statistical optimality for computational gains, leading to suboptimal regret bounds like $O(T^{2/3})$ [Garber and Kretzu, 2023; Wan et al., 2022].

Contributions. We propose LightONS (Light Online Newton Step), a simple variant of ONS that substantially reduces the runtime while retaining the optimal regret. Our method preserves the elegant structure of ONS and inherits its applicability to various scenarios. Our contributions include:

- **An optimal and efficient algorithm for OXO.** As summarized in Table 1, LightONS attains the best-known total runtime $O(d^2T + d^\omega \sqrt{T \log T})$ to achieve the minimax-optimal regret $O(d \log T)$ for OXO. In terms of regret, LightONS matches ONS's dependence on all problem parameters in OXO (T, d, D, G, α), whereas the prior method [Mhammedi and Gatmiry, 2023] suffers from large multiplicative constants. Empirical validations in Appendix A also corroborate the theoretical superiority of our method.

- **An optimal and efficient algorithm for SXO.** Equipped with the online-to-batch conversion, LightONS yields (up to poly-logarithmic factors) the optimal sample complexity $T = \widetilde{O}(d/\varepsilon)$ for an excess risk of ε, thus reducing the total runtime to $\widetilde{O}(d^3/\varepsilon)$ when $\varepsilon = O(1/d)$. Our result answers a COLT'13 open problem of Koren [2013]. We further provide evidence that the runtime $\widetilde{O}(d^3/\varepsilon)$ is unlikely to be improved in practice.

- **Applicability across various scenarios.** Importantly, LightONS preserves the online mirror descent (OMD) framework of ONS and inherits ONS's structural flexibility, especially in accommodating diverse local norms. In Section 5, we demonstrate LightONS smoothly replaces ONS and reduces runtime in various scenarios, including gradient-norm adaptivity, parametric stochastic bandits, and memory-efficient OXO. In contrast, the prior method [Mhammedi and Gatmiry, 2023], tailored for OXO, lacks the flexibility to fit these scenarios.

Outline. The remainder of this paper is organized as follows. Section 2 presents preliminaries. Section 3 introduces our method and key analytical ingredients. Section 4 discusses the implications of LightONS to SXO, which answers a COLT'13 open problem. Section 5 demonstrates the broad applicability of LightONS inherited from ONS. Section 6 concludes the paper and discusses future directions. Omitted technical details are deferred to the appendices.

2. Preliminaries

In this section, we introduce notations used throughout this paper, formalize the problem setting of Online eXp-concave Optimization (OXO), and review key relevant prior work.

Notations. Let $[a]_+ = \max\{0, a\}$, $[N] = \{1, \dots, N\}$, and $\mathcal{B}(R) = \{\mathbf{x} \mid \|\mathbf{x}\|_2 \le R\}$. For a positive-definite and symmetric matrix M, let $\lambda_i(M)$ be its i-th greatest eigenvalue and let $\|\mathbf{x}\|_M = \sqrt{\mathbf{x}^\intercal M \mathbf{x}}$ be the Mahalanobis norm. Let $\Pi_{\mathcal{X}}^M[\mathbf{y}] = \arg\min_{\mathbf{x}\in\mathcal{X}} \|\mathbf{x} - \mathbf{y}\|_M^2$ be the Mahalanobis projection, which exists and is unique for a convex and compact domain $\mathcal{X}$ [Boyd and Vandenberghe, 2004], and let $\Pi_{\mathcal{X}}[\mathbf{y}] = \Pi_{\mathcal{X}}^I[\mathbf{y}] = \arg\min_{\mathbf{x}\in\mathcal{X}} \|\mathbf{x} - \mathbf{y}\|_2^2$ be the Euclidean projection. We write $\mathsf{EP}_{\mathcal{X}}$ and $\mathsf{MP}_{\mathcal{X}}$ for the runtime of Euclidean and Mahalanobis projection onto $\mathcal{X}$, respectively.

2.1. Problem Setting

Online Convex Optimization (OCO) unfolds as a game between a learner and an environment over T rounds. At each round $t \in [T]$, the learner selects a decision $\mathbf{x}_t$ from a compact convex domain $\mathcal{X} \subseteq \mathbb{R}^d$, and the environment simultaneously reveals a convex loss function $f_t : \mathcal{X} \to \mathbb{R}$; then the learner incurs a loss $f_t(\mathbf{x}_t)$ and observes a gradient $\nabla f_t(\mathbf{x}_t)$ for updates. The performance of the learner is measured by its *regret* against some comparator $\mathbf{u} \in \mathcal{X}$, which is defined as:

$$\textsc{Reg}_T(\mathbf{u}) = \sum_{t=1}^T f_t(\mathbf{x}_t) - \sum_{t=1}^T f_t(\mathbf{u}).$$

The definition of exp-concavity [Kivinen and Warmuth, 1999; Cesa-Bianchi and Lugosi, 2006] and standard regularity assumptions of OXO [Hazan, 2016; Hazan et al., 2007; Mhammedi and Gatmiry, 2023] are formally stated below.

Assumption 1 (bounded domain) *The domain $\mathcal{X} \subseteq \mathbb{R}^d$ is convex and compact, and has a diameter of D, i.e., $\max_{(\mathbf{x},\mathbf{y})\in\mathcal{X}^2} \|\mathbf{x} - \mathbf{y}\|_2 \le D$. For technical convenience, we further assume that $\max_{\mathbf{x}\in\mathcal{X}} \|\mathbf{x}\|_2 \le D/2$.*

Assumption 2 (bounded gradient) *For any $t \in [T]$, the loss function $f_t : \mathcal{X} \to \mathbb{R}$ is differentiable and G-Lipschitz on $\mathcal{X}$, i.e., $\max_{\mathbf{x}\in\mathcal{X}} \|\nabla f_t(\mathbf{x})\|_2 \le G$.*

Assumption 3 (exp-concave loss) *For any $t \in [T]$, the loss function $f_t : \mathcal{X} \to \mathbb{R}$ is α-exp-concave on $\mathcal{X}$, i.e., $\exp(-\alpha f_t(\cdot))$ is concave on $\mathcal{X}$.*

2.2. Important Progress

In this subsection, we review two important OXO algorithms, namely Online Newton Step (ONS) [Hazan et al., 2007] and Online Quasi-Newton Step (OQNS) [Mhammedi and Gatmiry, 2023].

Online Newton Step (ONS). Algorithm 1 summarizes the update of ONS and enjoys the following theoretical guarantees on its regret and runtime.

Proposition 1 (Theorem 2 of Hazan et al. [2007]) *Under Assumptions 1–3, ONS (Algorithm 1) with $\gamma_0 = \frac{1}{2}\min\{\frac{1}{DG}, \alpha\}$ satisfies that, for any $\mathbf{u} \in \mathcal{X}$,*

$$\textsc{Reg}_T(\mathbf{u}) \le \frac{d}{2\gamma_0} \log\left(1 + \frac{G^2}{d\epsilon} T\right) + \frac{\gamma_0 \epsilon D^2}{8}, \tag{1a}$$

$$\textsc{Runtime} \le O((d^2 + \mathsf{MP}_{\mathcal{X}})T). \tag{1b}$$

ONS follows the classical framework of online mirror descent (OMD) [Orabona, 2019], equipped with a time-varying Mahalanobis norm induced by the Hessian-related matrix A_t. The computational bottleneck of ONS lies in Line 6 of Algorithm 1, the Mahalanobis projection $\mathbf{x}_{t+1} = \Pi_{\mathcal{X}}^{A_t}[\widehat{\mathbf{x}}_{t+1}]$. Once the decision $\widehat{\mathbf{x}}_{t+1}$ exits the domain $\mathcal{X}$, it must be projected back immediately. Since each Mahalanobis projection takes $\widetilde{O}(d^\omega)$ arithmetic operations and ONS projects in $O(T)$ rounds in the worst case, the crippling total runtime $\widetilde{O}(d^\omega T)$ emerges, which evaluates to $\widetilde{O}(d^3 T)$ in practice.

Hazan et al. [2007] introduce the following lemma to characterize the curvature induced by exp-concavity, which is crucial to ONS and other practical OXO algorithms. This lemma is essentially Lemma 3 of Hazan et al. [2007], with improved constants. Its proof is deferred to Appendix B.1.

Lemma 1 *If a function $f : \mathcal{X} \to \mathbb{R}$ is α-exp-concave and differentiable, then for any $(\mathbf{x}, \mathbf{u}) \in \mathcal{X}^2$, $D \geq \|\mathbf{x} - \mathbf{u}\|_2$, $G \geq \|\nabla f(\mathbf{x})\|_2$, $\gamma \leq \gamma_0 = \frac{1}{2}\min\{\frac{1}{DG}, \alpha\}$, it holds that*

$$f(\mathbf{x}) - f(\mathbf{u}) \leq \nabla f(\mathbf{x})^\mathsf{T}(\mathbf{x} - \mathbf{u}) - \frac{\gamma}{2}\left(\nabla f(\mathbf{x})^\mathsf{T}(\mathbf{x} - \mathbf{u})\right)^2. \tag{2}$$

We remark that the preceding lemma necessitates the bounded domain assumption (Assumption 1) to achieve the optimal regret $O(d \log T)$. In particular, a finite diameter D is always required to obtain a non-zero curvature parameter γ, which reflects the curvature induced by exp-concavity.

Online Quasi-Newton Step (OQNS). OQNS [Mhammedi and Gatmiry, 2023] effectively reduces the runtime to reach the optimal asymptotic regret $O(d \log T)$, as the following proposition states.

Proposition 2 (Theorem 9 of Mhammedi and Gatmiry [2023]) *Under Assumptions 1–3, OQNS (Algorithm 3 of Mhammedi and Gatmiry [2023]) with $\gamma = \frac{1}{2}\min\{\frac{1}{2DG}, \alpha\}$ satisfies that, for any $\mathbf{u} \in \mathcal{X}$,*

$$\textsc{Reg}_T(\mathbf{u}) \leq \frac{5d}{\gamma}\log(d + T) + \frac{11DGd}{2}\log T + 3DGd, \tag{3a}$$

$$\textsc{Runtime} \leq O\big((\mathsf{EP}_{\mathcal{X}} + d^2 \log T)T + d^\omega\sqrt{T \log T}\big). \tag{3b}$$

OQNS eliminates Mahalanobis projections with a log-barrier, such as $\log\frac{1}{1-\|\mathbf{x}\|_2^2}$ for $\mathcal{X} = \mathcal{B}(1)$, shifting the computational burden to Hessian-inverse evaluations. Key components of OQNS are illustrated below, with $\mathcal{X} = \mathcal{B}(1)$ and $G = 1$ as in [Mhammedi and Gatmiry, 2023].

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \text{Approx}\big((\nabla^2\Phi_t(\mathbf{x}_t))^{-1}\nabla\Phi_t(\mathbf{x}_t)\big), \quad \text{where}$$
$$\Phi_t(\mathbf{x}) \triangleq \eta d \log\frac{1}{1 - \|\mathbf{x}\|_2^2} + \frac{d + \eta}{2}\|\mathbf{x}\|_2^2 + \sum_{s=1}^{t}\left(\nabla f_s(\mathbf{x}_s)^\mathsf{T}\mathbf{x} + \frac{\gamma}{2}\big(\nabla f_s(\mathbf{x}_s)^\mathsf{T}(\mathbf{x} - \mathbf{x}_s)\big)^2\right). \tag{4}$$

Evaluating the Hessian-inverse still takes $O(d^\omega)$ time and OQNS mitigates this issue by approximating the Hessian-inverse gradient product with incremental updates. The approximation procedure, $\text{Approx}(\cdot)$, returns within $O(d^2 \log T)$ time under proper conditions. OQNS controls the number of exact Hessian-inverse evaluations to $O(\sqrt{T \log T})$, leading to a total runtime of $O(d^2 T \log T + d^\omega\sqrt{T \log T})$, while achieving the optimal regret $O(d \log T)$.

However, their improvement introduces large constant factors. Specifically, when $\alpha \geq \frac{1}{DG}$, the leading term of ONS, $\log T$, carries coefficient DGd in Eq. (1a), whereas OQNS's leading coefficient is $\frac{51}{2}DGd$ in Eq. (3a). Moreover, OQNS departs from the OMD framework of ONS, limiting its adaptability to broader scenarios, which will be discussed in Section 5.

Algorithm 1 ONS [Hazan et al., 2007]

Input: preconditioner coefficient ϵ.

1: Initialize $\gamma_0 = \frac{1}{2}\min\{\frac{1}{DG}, \alpha\}$, $A_0 = \epsilon I$, $\mathbf{x}_1 = \mathbf{0}$.
2: **for** $t = 1, \dots, T$ **do**
3: Observe $\nabla f_t(\mathbf{x}_t)$.
4: $A_t = A_{t-1} + \nabla f_t(\mathbf{x}_t)\nabla f_t(\mathbf{x}_t)^\mathsf{T}$.
5: $\widehat{\mathbf{x}}_{t+1} = \mathbf{x}_t - \frac{1}{\gamma_0} A_t^{-1}\nabla f_t(\mathbf{x}_t)$.
6: $\mathbf{x}_{t+1} = \Pi_{\mathcal{X}}^{A_t}[\widehat{\mathbf{x}}_{t+1}]$.
7: **end for**

Algorithm 2 LightONS.Core

Input: preconditioner coef. ϵ, deferral coef. k.

1: Initialize $\gamma = \frac{1}{2}\min\{\frac{2}{(k+1)DG}, \alpha\}$, $A_0 = \epsilon I$, $\mathbf{x}_1 = \mathbf{0}$.
2: **for** $t = 1, \dots, T$ **do**
3: Observe $\nabla f_t(\mathbf{x}_t)$.
4: $A_t = A_{t-1} + \nabla f_t(\mathbf{x}_t)\nabla f_t(\mathbf{x}_t)^\mathsf{T}$.
5: $\widehat{\mathbf{x}}_{t+1} = \mathbf{x}_t - \frac{1}{\gamma} A_t^{-1}\nabla f_t(\mathbf{x}_t)$.
6: $\mathbf{x}_{t+1} = \begin{cases} \widehat{\mathbf{x}}_{t+1} & \text{if } \|\widehat{\mathbf{x}}_{t+1}\|_2 \le kD/2 \\ \Pi_{\mathcal{X}}^{A_t}[\widehat{\mathbf{x}}_{t+1}] & \text{otherwise} \end{cases}$.
7: **end for**

3. Our Algorithm: LightONS

Prior progress naturally raises a question: Can we retain the optimality and simplicity of ONS, while achieving a total runtime that is competitive with, or even superior to, the state-of-the-art? Our work is motivated by answering this question in the affirmative.

In this spirit, we present our algorithm LightONS. In Section 3.1, to illustrate the key idea, we introduce the core algorithm that only differs from ONS by one line of code. However, the core algorithm is essentially *improper learning*. To address this issue, we introduce the improper-to-proper conversion in Section 3.2, which yields the complete version of LightONS.

Furthermore, we empirically validate the superiority of our method, which corroborates theoretical guarantees. Details of the experiments are deferred to Appendix A.

3.1. Amortizing Deferred Projections

We first introduce the LightONS.Core in Algorithm 2, which amortizes the costly Mahalanobis projections with a *deferred-projection* mechanism. LightONS.Core only differs from ONS by one line of code, as shown in Line 6 of Algorithms 1 and 2. While ONS projects onto $\mathcal{X}$ immediately when the decision exits $\mathcal{X}$, LightONS.Core continues to update without projection outside $\mathcal{X}$, and projects only when the decision exits an *expanded* domain $\widetilde{\mathcal{X}}_k \subseteq \mathbb{R}^d$ defined as

$$\widetilde{\mathcal{X}}_k \triangleq \mathcal{B}(kD/2) = \{\mathbf{x} \in \mathbb{R}^d \mid \|\mathbf{x}\|_2 \le kD/2\}, \tag{5}$$

where $k > 1$ is the deferral coefficient. Essentially, $\widetilde{\mathcal{X}}_k$ is obtained by scaling a Euclidean ball that contains $\mathcal{X}$ by a factor of $k > 1$.

Clearly, a larger k implies lower runtime, as it invokes fewer Mahalanobis projections. The following lemma quantifies the relationship between the deferral coefficient k and the number of Mahalanobis projections N. Its proof is deferred to Appendix C.1.

Lemma 2 *Under Assumption 1, and that the loss functions $\{f_t\}_{t=1}^T$ are α-exp-concave, differentiable and G-Lipschitz on $\widetilde{\mathcal{X}}_k$, let N denote the number of Mahalanobis projections in LightONS.Core (Algorithm 2) over T rounds, then*

$$N \le \left\lfloor \frac{2}{(k-1)D\gamma}\sqrt{\frac{d}{\epsilon}T} \right\rfloor. \tag{6}$$

Algorithm 3 LightONS

Input: preconditioner coefficient ϵ, deferral coefficient k.

Initialize $\gamma' = \frac{1}{2}\min\{\frac{1}{DG}, \alpha, \frac{4}{(k+1)c_f c_g DG}\}$, $A_0 = \epsilon I$, $\mathbf{x}_1 = \mathbf{y}_1 = \mathbf{0}$.
for $t = 1, \dots, T$ **do**
  Observe $\nabla f_t(\mathbf{x}_t)$; and construct $\nabla g_t(\mathbf{y}_t)$ satisfying Condition 1.
  $A_t = A_{t-1} + \boldsymbol{\nabla}_t \boldsymbol{\nabla}_t^\intercal$, where $\boldsymbol{\nabla}_t = c_f \nabla g_t(\mathbf{y}_t)$.
  $\widehat{\mathbf{y}}_{t+1} = \mathbf{y}_t - \frac{1}{\gamma'} A_t^{-1} \boldsymbol{\nabla}_t$.
  $\mathbf{y}_{t+1} = \begin{cases} \widehat{\mathbf{y}}_{t+1} & \text{if } \|\widehat{\mathbf{y}}_{t+1}\|_2 \le kD/2 \\ \Pi^{A_t}_{\mathcal{B}(D/2)}[\widehat{\mathbf{y}}_{t+1}] & \text{otherwise} \end{cases}$.
  $\mathbf{x}_{t+1} = \Pi_{\mathcal{X}}[\mathbf{y}_{t+1}]$.
end for

On the other hand, a larger k exacerbates the deviation of LightONS.Core from ONS, potentially harming the regret guarantee. In particular, increasing k degrades the curvature parameter γ, jeopardizing the curvature benefit of exp-concavity. In the extreme case where k approaches infinity, γ collapses to zero, and exp-concavity degenerates to mere convexity. The following theorem establishes the theoretical guarantees of LightONS.Core, revealing the trade-off between regret and runtime induced by the deferral coefficient k. Its proof is deferred to Appendix C.2.

Theorem 1 *Under Assumption 1, and that the loss functions $\{f_t\}_{t=1}^T$ are α-exp-concave, differentiable and G-Lipschitz on $\widetilde{\mathcal{X}}_k$, with $\gamma = \frac{1}{2}\min\{\frac{2}{(k+1)DG}, \alpha\}$, LightONS.Core (Algorithm 2) satisfies that, for any $\mathbf{u} \in \mathcal{X}$,*

$$\textsc{Reg}_T(\mathbf{u}) \le \frac{d}{2\gamma}\log\left(1 + \frac{G^2}{d\epsilon}T\right) + \frac{\gamma\epsilon D^2}{8}, \tag{7a}$$

$$\textsc{Runtime} \le O\big(d^2 T + (k-1)^{-1}\sqrt{dT/\epsilon}\cdot \texttt{MP}_{\mathcal{X}}\big). \tag{7b}$$

Trade-off of the deferral coefficient k. When k increases, the regret bound in Eq. (7a) grows as $O(\frac{1}{\gamma}) = O(k+1)$, whereas the time spent on Mahalanobis projections in Eq. (7b) decreases as $O(N) = O(\frac{1}{k-1})$. With $k = 2$, LightONS.Core already achieves a significant runtime improvement over ONS without sacrificing the optimal regret. Specifically, the regret bound grows by at most a factor of $\frac{3}{2}$, while the number of projections is substantially reduced from $O(T)$ to $O(\sqrt{T})$.

Improper learning issue. Unfortunately, LightONS.Core falls in the scope of *improper learning* [Shalev-Shwartz and Ben-David, 2014], as the algorithm's decisions $\mathbf{x}_t \in \widetilde{\mathcal{X}}_k \supset \mathcal{X}$ may reside beyond the domain while the comparator $\mathbf{u} \in \mathcal{X}$ is strictly constrained to the domain. Moreover, LightONS.Core requires additional assumptions that the Lipschitzness and exp-concavity of the loss functions extend to the expanded domain $\widetilde{\mathcal{X}}_k$. Vitally, improper learning suppresses the theoretical performance limits of its proper counterparts. A notable illustration is online logistic regression, where an improper learner achieves the regret bound of $O(d\log(GT))$ [Foster et al., 2018], while proper learners are limited to a regret lower bound of $\Omega(de^G \log T)$ [Hazan et al., 2014].

3.2. The Improper-to-Proper Conversion

We introduce LightONS in Algorithm 3, which inherits the favorable regret-runtime trade-off of LightONS.Core while ensuring proper learning over the domain $\mathcal{X}$, thanks to domain-conversion

techniques from parameter-free online learning [Cutkosky and Orabona, 2018; Cutkosky, 2020]. The improper-to-proper conversion is conceptually straightforward yet technically subtle, as the surrogate loss preserves the curvature benefit of exp-concavity even though it is not exp-concave.

The conversion works by constructing surrogate losses and projecting surrogate decisions. At each round, LightONS constructs a surrogate loss $g_t : \mathbb{R}^d \to \mathbb{R}$ and supplies g_t to an underlying LightONS.Core; then the underlying LightONS.Core outputs a surrogate decision $\mathbf{y}_t \in \widetilde{\mathcal{X}}_k$, which LightONS maps to a proper true decision $\mathbf{x}_t \in \mathcal{X}$ via a Euclidean projection.

Any surrogate loss satisfying the condition below guarantees a valid conversion.

Condition 1 *For some $c_f \geq 1$ and $c_g \geq 1$, the surrogate loss function $g_t : \mathbb{R}^d \to \mathbb{R}$ satisfies that, for any $\mathbf{u} \in \mathcal{X}$, $\|\nabla g_t(\mathbf{y}_t)\|_2 \leq c_g \|\nabla f_t(\mathbf{x}_t)\|_2$ and $\nabla f_t(\mathbf{x}_t)^\mathsf{T}(\mathbf{x}_t - \mathbf{u}) \leq c_f \nabla g_t(\mathbf{y}_t)^\mathsf{T}(\mathbf{y}_t - \mathbf{u})$.*

We note that two such conversions have been proposed in the literature. The first, by Cutkosky and Orabona [2018], achieves $c_f = 2$ and $c_g = 1$. Later, an improved conversion by Cutkosky [2020] achieves $c_f = c_g = 1$. We prefer the latter which yields smaller constants and tighter regret.

Lemma 3 (Theorem 2 of Cutkosky [2020]) *Under Assumptions 1 and 2, let $\mathbf{x}_t = \Pi_{\mathcal{X}}[\mathbf{y}_t]$, the surrogate loss function $g_t : \mathbb{R}^d \to \mathbb{R}$ and its subgradient at $\mathbf{y}_t$ are defined as follows:*

$$g_t(\mathbf{y}) \triangleq \nabla f_t(\mathbf{x}_t)^\mathsf{T}\mathbf{y} + \frac{[-\nabla f_t(\mathbf{x}_t)^\mathsf{T}(\mathbf{y}_t - \mathbf{x}_t)]_+}{\|\mathbf{y}_t - \mathbf{x}_t\|_2} \|\mathbf{y} - \Pi_{\mathcal{X}}[\mathbf{y}]\|_2 , \tag{8a}$$

$$\nabla g_t(\mathbf{y}_t) = \nabla f_t(\mathbf{x}_t) + \frac{[-\nabla f_t(\mathbf{x}_t)^\mathsf{T}(\mathbf{y}_t - \mathbf{x}_t)]_+}{\|\mathbf{y}_t - \mathbf{x}_t\|_2^2} (\mathbf{y}_t - \mathbf{x}_t) , \tag{8b}$$

then for any $\mathbf{u} \in \mathcal{X}$, $\|\nabla g_t(\mathbf{y}_t)\|_2 \leq \|\nabla f_t(\mathbf{x}_t)\|_2$, and $\nabla f_t(\mathbf{x}_t)^\mathsf{T}(\mathbf{x}_t - \mathbf{u}) \leq \nabla g_t(\mathbf{y}_t)^\mathsf{T}(\mathbf{y}_t - \mathbf{u})$.

Side effects of the conversion. The computational overhead of the conversion is negligible. In each round, Eq. (8b) constructs the surrogate gradient $\nabla g_t(\mathbf{y}_t)$ with only $O(d)$ time apart from the Euclidean projection, dominated by the $O(d^2)$ cost of updating A_t and A_t^{-1} (see Appendix B.3).

A notable observation is that the surrogate loss g_t in Lemma 3 is *not* exp-concave, seemingly precluding the curvature benefit. Fortunately, the following lemma shows that the surrogate loss g_t inherits the curvature in a form closely mirroring Lemma 1, with proof in Appendix C.3.

Lemma 4 *Under Assumptions 1–3, with $\gamma' = \frac{1}{2}\min\{\frac{1}{DG}, \alpha, \frac{4}{(k+1)c_f c_g DG}\}$, for any $\mathbf{u} \in \mathcal{X}$ and any surrogate loss function g_t satisfying Condition 1, let $\boldsymbol{\nabla}_t = c_f \nabla g_t(\mathbf{y}_t)$, then*

$$\begin{aligned} f_t(\mathbf{x}_t) - f_t(\mathbf{u}) &\leq \nabla f_t(\mathbf{x}_t)^\mathsf{T}(\mathbf{x}_t - \mathbf{u}) - \frac{\gamma_0}{2}\left(\nabla f_t(\mathbf{x}_t)^\mathsf{T}(\mathbf{x}_t - \mathbf{u})\right)^2 \\ &\leq \boldsymbol{\nabla}_t^\mathsf{T}(\mathbf{y}_t - \mathbf{u}) - \frac{\gamma'}{2}\left(\boldsymbol{\nabla}_t^\mathsf{T}(\mathbf{y}_t - \mathbf{u})\right)^2 . \end{aligned} \tag{9}$$

We remark that, when the deferral coefficient $k \leq 3$ and $c_f = c_g = 1$ (e.g., in Lemma 3), the surrogate curvature parameter γ' in Lemma 4 is *unimpaired* relative to the original γ_0 in Lemma 1.

It is worth noting that prior work has also employed this domain-conversion technique to address projection-related issues, though for different purposes, including non-stationary online learning [Zhao et al., 2025] and universal online learning [Yang et al., 2024].

The following theorem establishes the theoretical guarantees of LightONS, and an immediate corollary specifies its regret and runtime by instantiating deferral coefficient $k = 2$ and the surrogate loss in Lemma 3. Its proof is deferred to Appendix C.4.

Theorem 2 *Under Assumptions 1–3, and that each surrogate gradient $\nabla g_t(\mathbf{y}_t)$ takes $O(\mathsf{EP}_{\mathcal{X}} + d)$ time, with $\gamma' = \frac{1}{2}\min\{\frac{1}{DG}, \alpha, \frac{4}{(k+1)c_f c_g DG}\}$, LightONS (Algorithm 3) satisfies that, for any $\mathbf{u} \in \mathcal{X}$,*

$$\textsc{Reg}_T(\mathbf{u}) \le \frac{d}{2\gamma'}\log\left(1 + \frac{c_f^2 c_g^2 G^2}{d\epsilon}T\right) + \frac{\gamma'\epsilon D^2}{8}, \tag{10a}$$

$$\textsc{Runtime} \le O\big((\mathsf{EP}_{\mathcal{X}} + d^2)T + (k-1)^{-1}d^{\omega+0.5}\sqrt{T/\epsilon}\log T\big). \tag{10b}$$

Corollary 1 *Under Assumptions 1–3, using the surrogate loss function in Lemma 3, $k = 2$, and $\epsilon = 1 + d\log T$, with $\gamma_0 = \frac{1}{2}\min\{\frac{1}{DG}, \alpha\}$, LightONS (Algorithm 3) satisfies that, for any $\mathbf{u} \in \mathcal{X}$,*

$$\textsc{Reg}_T(\mathbf{u}) \le \frac{d}{2\gamma_0}\log\left(1 + \frac{G^2}{d\epsilon}T\right) + \frac{\gamma_0\epsilon D^2}{8}, \tag{11a}$$

$$\textsc{Runtime} \le O\big((\mathsf{EP}_{\mathcal{X}} + d^2)T + d^{\omega}\sqrt{T\log T}\big). \tag{11b}$$

Remark 1 *LightONS achieves a runtime better than both ONS and OQNS, at the cost of a regret slightly worse than ONS, as seen in the second inequality in Eq. (9). Nonetheless, this degradation is dominated by the problem-dependent parameters of OXO, namely T, d, D, G, α. In their dependence on these parameters, LightONS's regret in Eq. (11a) exactly matches ONS's regret in Eq. (1a).*

Remark 2 *LightONS retains the flexible OMD framework of ONS. In fact, LightONS differs from ONS only in two aspects:* (i) *the deferred-projection mechanism (introduced in LightONS.Core to boost efficiency) and* (ii) *the improper-to-proper conversion (introduced here to ensure proper learning), both of which are largely* orthogonal *to the mirror-descent update in the standard ONS. Consequently, LightONS applies to various scenarios where ONS is essential, particularly when its mirror-descent update plays a critical role. We illustrate these applications in Section 5.*

Numerical implementation. The runtime in Eq. (11b) arises from efficient numerical primitives, detailed in Appendix B.3. The term $O(d^2T)$ reflects rank-one updates of A_t^{-1}, which avoids the $O(d^{\omega})$ cost of inverting A_t from scratch. The term $O(d^{\omega}\sqrt{T\log T})$ accounts for the infrequent Mahalanobis projections onto $\mathcal{B}(D/2)$: each projection reduces to a one-dimensional root-finding problem solvable by bisection in $O(d^{\omega}\log T)$ time, and the deferred-projection mechanism caps the total number of such projections, so that their cumulative cost remains sublinear in T.

4. Answering a COLT'13 Open Problem

Via the online-to-batch conversion, our method applies to Stochastic eXp-concave Optimization (SXO), where the optimal OXO regret translates into the (near) optimal SXO sample complexity while substantially reducing the computational cost. This extension answers a COLT'13 open problem posed by Koren [2013], demonstrating our method's significance beyond OXO.

4.1. Restatement of the Open Problem

SXO seeks to minimize an exp-concave function $F : \mathcal{X} \to \mathbb{R}$ over a convex domain $\mathcal{X} \subseteq \mathbb{R}^d$, and the learner has access to F only through some stochastic oracle. In this section, we consider the case where F is the expectation of a random function $f : \mathcal{X} \times \Xi \to \mathbb{R}$ over a distribution on Ξ, and the

learner has access to the gradient of $f(\cdot;\xi)$ with ξ drawn from $\mathcal{D}_\xi$. We are interested in the sample complexity and total runtime required to find an ε-optimal solution $\mathbf{x}_\varepsilon$, i.e.,

$$F(\mathbf{x}_\varepsilon) - \min_{\mathbf{x}\in\mathcal{X}} F(\mathbf{x}) \le \varepsilon, \quad \text{where } F(\mathbf{x}) = \mathop{\mathbb{E}}_{\xi\sim\mathcal{D}_\xi}[f(\mathbf{x};\xi)].$$

The regularity of the random function is formally stated below.

Assumption 4 *The loss function $F:\mathcal{X}\to\mathbb{R}$ is the expectation of a random function $f:\mathcal{X}\times\Xi\to\mathbb{R}$ over an unknown distribution $\mathcal{D}_\xi$ on Ξ, i.e., $F(\mathbf{x}) = \mathbb{E}_{\xi\sim\mathcal{D}_\xi}[f(\mathbf{x};\xi)]$. For any $\xi\in\Xi$, the random loss function $f(\cdot;\xi)$ is α-exp-concave, differentiable and G-Lipschitz over $\mathcal{X}$. For any query point $\mathbf{x}\in\mathcal{X}$, the stochastic gradient oracle returns $\nabla f(\mathbf{x};\xi)$ with ξ i.i.d. drawn from $\mathcal{D}_\xi$.*

Koren [2013] notes that, with the online-to-batch conversion, ONS's $O(d\log T)$ regret for OXO implies a sample complexity of $\widetilde{O}(d/\varepsilon)$ and a total runtime of $\widetilde{O}(d^4/\varepsilon)$ for SXO, [2] where $\widetilde{O}(\cdot)$ hides poly-logarithmic factors in d/ε. This quartic dependence on d is even more pronounced in comparison to stochastic strongly convex optimization, where Online Gradient Descent (OGD) implies a total runtime $\widetilde{O}(d/\varepsilon)$ [Hazan and Kale, 2011], motivating the following open problem.

Open problem [Koren, 2013]. Under Assumption 4 and that $\mathcal{X}=\mathcal{B}(1)$,

(a) Is it possible to find an SXO algorithm that attains the sample complexity of $\widetilde{O}(d/\varepsilon)$ with only linear-in-d runtime per iteration, i.e., $\widetilde{O}(d^2/\varepsilon)$ runtime overall?

(b) Is it possible to perform any better than $\widetilde{O}(d^4/\varepsilon)$ runtime overall?

The first part of the open problem remains open. In particular, Mahdavi et al. [2015] establish an information-theoretic sample complexity lower bound of $\Omega(d/\varepsilon)$ for SXO, which implies a runtime lower bound of $\Omega(d^2/\varepsilon)$ since each gradient query costs $\Omega(d)$ time.

Section 4.2 answers the second part in the affirmative by combining LightONS with the online-to-batch conversion, obtaining total runtime $\widetilde{O}(d^3/\varepsilon)$ while maintaining the optimal $\widetilde{O}(d/\varepsilon)$ sample complexity. In Section 4.3, we further present evidence that this $\widetilde{O}(d^3/\varepsilon)$ runtime is likely unimprovable. If confirmed, it would refute the first part and settle the full open problem.

4.2. Answering the Open Problem with LightONS

Built upon the online-to-batch conversion of Mehta [2017], the LightONS-based SXO method reduces the total runtime to $\widetilde{O}(d^3/\varepsilon)$ for $\varepsilon = O(1/d)$. It is reasonable to only consider $\varepsilon = O(1/d)$. Indeed, when $\varepsilon = \Omega(1/d)$, the OGD-based SXO method achieves both a better sample complexity of $T=\widetilde{O}(1/\varepsilon^2)$ and a better runtime of $\widetilde{O}(d/\varepsilon^2)$. The LightONS-based SXO method also achieves the (near) optimal convergence rate both with high probability and in expectation, as the following theorem shows. Its proof is deferred to Appendix D.1.

Theorem 3 *Under Assumption 4 and that $\mathcal{X}=\mathcal{B}(1)$, let LightONS (Algorithm 3) run for T rounds with gradients $\{\nabla f(\mathbf{x}_t;\xi_t)\}_{t=1}^T$ where $\{\mathbf{x}_t\}_{t=1}^T$ are decisions, let $\bar{\mathbf{x}}_T = \frac{1}{T}\sum_{t=1}^T \mathbf{x}_t$, then*

2. From a theoretical perspective, the total runtime of applying ONS to SXO is $\widetilde{O}(d^{\omega+1}/\varepsilon)$, as discussed in Section 1. We adopt $\omega=3$ following the statement of the open problem [Koren, 2013], with emphasis on implementability.

Table 2: Regret and total runtime comparison between ONS and LightONS on three benchmark applications. "OQNS?" indicates whether OQNS supports the same application.

Application	**Regret (ONS)**	**Runtime (ONS)**	**Regret (LightONS)**	**Runtime (LightONS)**	**OQNS?**
Gradient-norm adaptivity	$O(d \log G_T)$	$\widetilde{O}(d^\omega T)$	same	$O(d^2 T + d^\omega \sqrt{T} \log T)$	hardly
Generalized linear bandits	$\widetilde{O}(d\sqrt{T} + \kappa d^2)$	$\widetilde{O}(d^2 KT + d^\omega T)$	same order	$\widetilde{O}(d^2 KT + d^\omega \cdot \min\{\sqrt{\kappa dT} \log \kappa, T\})$	hardly
Memory-efficient OXO	$O(d' \log T)$	$\widetilde{O}(d^\omega T)$	same	$\widetilde{O}(d' dT + d^\omega \sqrt{T})$	intricate

- *For any* $\delta \in (0,1)$*, with probability at least* $1-\delta$*, when* $T = \Theta(\frac{d}{\varepsilon} \log \frac{d}{\varepsilon} \log \frac{1}{\delta})$, $F(\bar{\mathbf{x}}_T) - \min_{\mathbf{x} \in \mathcal{X}} F(\mathbf{x}) \le O(\varepsilon)$ *and* RUNTIME $\le \widetilde{O}(\frac{d^3}{\varepsilon} + \frac{d^{3.5}}{\sqrt{\varepsilon}})$.
- *When* $T' = \Theta(\frac{d}{\varepsilon} \log \frac{d}{\varepsilon})$, $\mathbb{E}[F(\bar{\mathbf{x}}_{T'}) - \min_{\mathbf{x} \in \mathcal{X}} F(\mathbf{x})] \le O(\varepsilon)$ *and* RUNTIME $\le \widetilde{O}(\frac{d^3}{\varepsilon} + \frac{d^{3.5}}{\sqrt{\varepsilon}})$.

4.3. Discussions on SXO

The LightONS-based SXO method in Theorem 3 matches the best-known SXO runtime up to poly-logarithmic factors. An online-to-batch conversion of OQNS also yields a total runtime of $\widetilde{O}(d^3/\varepsilon)$ but carries significant practical overheads due to its large constant factors, as discussed in Section 3. Our experiments in Appendix A, conducted under SXO settings, confirm this difference.

We conjecture that no SXO algorithm can asymptotically beat total runtime $\widetilde{O}(d^3/\varepsilon)$. A natural alternative approach to SXO is empirical risk minimization (ERM) [Koren and Levy, 2015; Mehta, 2017], which reduces SXO to offline exp-concave optimization and admits offline acceleration techniques. The following two observations indicate that ERM-based SXO methods are unlikely to surpass the $\widetilde{O}(d^3/\varepsilon)$ runtime barrier, even when these methods use more working memory $\Omega(d^2 + d/\varepsilon)$ to store all samples, whereas the OXO-based methods require only $O(d^2)$ memory.

- **Fast matrix multiplication.** Linear regression with random design, a special case of SXO, reduces to solving a linear system with d variables and $\widetilde{O}(d/\varepsilon)$ equations. Although fast matrix multiplication accelerates this to $\widetilde{O}(d^\omega/\varepsilon)$ runtime [Ibarra et al., 1982], this runtime does not plausibly extend beyond linear regression and reverts to $\widetilde{O}(d^3/\varepsilon)$ in practice.
- **Cutting-plane methods.** The best-known cutting-plane methods of Lee et al. [2015]; Jiang et al. [2020], as detailed in Appendix D.2, solve the offline exp-concave optimization problem to $O(\varepsilon)$-accuracy in $\widetilde{O}(d^3/\varepsilon)$ time. We remark that cutting-plane methods further assume well-roundedness of the domain, which may bring additional computational overhead.

Moreover, we give an intuitive explanation of the runtime barrier $\widetilde{O}(d^3/\varepsilon)$ in Appendix D.2.

5. Applications to Various Problems

ONS extends its influence far beyond the classical settings of OXO and SXO. Since LightONS is designed to retain core updates of ONS with minimal modifications, it preserves its elegant structure.

This enables seamless integration into a range of applications where ONS serves as the computational core, not only maintaining its statistical advantages but also significantly enhancing efficiency.

We highlight three representative applications of ONS (gradient-norm adaptivity, parametric stochastic bandits, and memory-efficient OXO) where LightONS seamlessly fits, preserving statistical benefits while requiring minimal additional analytical effort. In contrast, deploying OQNS in these settings is either infeasible or would require substantial and non-trivial modifications. The main results are summarized in Table 2, comparing ONS with LightONS and OQNS. Due to space constraints, we defer details and proofs to Appendix E.

5.1. Gradient-Norm Adaptivity

As shown by Orabona et al. [2012], ONS achieves the following *problem-dependent* regret bound that scales with the accumulated squared gradient norms G_T instead of the time horizon T:

$$\textsc{Reg}_T(\mathbf{u}) = O\left(d \log G_T\right), \quad \text{where } G_T \triangleq \sum_{t=1}^{T} \|\nabla f_t(\mathbf{x}_t)\|_2^2, \tag{12}$$

where G_T can be $o(T)$ in benign environments, yielding regret far below worst-case bounds that scale with T. Yet, since $G_T \leq G^2 T$, the gradient-norm adaptive bound safeguards the optimal regret $O(d \log T)$ against worst cases. Prior work has leveraged Eq. (12) to achieve strong guarantees in various settings, including small-loss adaptivity for OXO with smoothness and comparator-norm adaptivity for unbounded OCO, as discussed below.

OXO with smoothness. Orabona et al. [2012] show that the gradient-norm adaptivity in Eq. (12) can be enhanced to small-loss adaptivity [Srebro et al., 2010; Zhao et al., 2020] in Eq. (13) under smoothness assumptions on loss functions $\{f_t\}_{t=1}^T$,

$$\textsc{Reg}_T = O\left(d \log L_T\right), \quad \text{where } L_T \triangleq \min_{\mathbf{u} \in \mathcal{X}} \sum_{t=1}^{T} \left(f_t(\mathbf{u}) - \min_{\mathbf{x} \in \mathcal{X}} f_t(\mathbf{x}) \right), \tag{13}$$

where the regret scales with the cumulative loss of the best comparator in hindsight.

Unbounded OCO. For OCO with unbounded domains, i.e., $\mathcal{X} = \mathbb{R}^d$, Cutkosky and Orabona [2018] achieve a comparator-norm adaptive regret bound without prior knowledge of the comparator norm $\|\mathbf{u}\|_2$ based on the gradient-norm adaptivity in Eq. (12),

$$\textsc{Reg}_T(\mathbf{u}) = \widetilde{O}\left(\|\mathbf{u}\|_2 \sqrt{d G_T} \right), \quad \text{for any unbounded comparator } \mathbf{u} \in \mathbb{R}^d. \tag{14}$$

Such guarantees for unbounded comparators are unachievable via classical OCO algorithms that explicitly depend on the domain diameter D, for example, OGD demands an explicit D to obtain $O(DG\sqrt{T})$ regret [Zinkevich, 2003; Abernethy et al., 2008].

Improvements by LightONS. Nevertheless, both [Orabona et al., 2012; Cutkosky and Orabona, 2018] incur the worst-case $\widetilde{O}(d^\omega T)$ runtime bottleneck of ONS. Replacing ONS in these pipelines, LightONS preserves the same regret bounds while improving the runtime to $O(d^2 T + d^\omega \sqrt{T} \log T)$.

In contrast, OQNS can hardly achieve gradient-norm adaptivity, let alone the small-loss adaptivity and comparator-norm adaptivity. Its computational efficiency relies on the log-barrier, which

inevitably introduces an $O(\log T)$ term which precludes adaptation to G_T. The OMD framework permits a flexible trade-off between stability and bias in the regret decomposition. While LightONS retains this flexibility, the log-barrier in OQNS overly suppresses the stability term, leading to a large bias term that does not scale with the gradient norms. Details are deferred to Appendix E.1.

5.2. Parametric Stochastic Bandits

Parametric stochastic bandits model decision-making problems with partial feedback, and generalized linear bandits (GLB) is a fundamental instance where the expected loss depends on an unknown parameter through a known link function. This formulation introduces non-linearities into linear bandits, enhancing their expressivity while retaining tractable solutions.

Current results. A notable challenge of GLB is the dependence on the condition number $\kappa \propto \exp(D)$ in regret bounds. Directly applying ONS to GLB yields an $\widetilde{O}(\kappa d\sqrt{T})$ regret [Jun et al., 2017], which becomes vacuous for $\kappa = \Omega(\sqrt{T})$. Recently, Zhang et al. [2025] propose the first *jointly efficient* GLB algorithm, which achieves an $\widetilde{O}(d\sqrt{T} + \kappa d^2)$ regret (statistically efficient) and is one-pass (computationally efficient). Their key technique is an ONS-based subroutine with carefully designed non-monotonic local norms, which enjoys the one-pass efficiency of ONS and eliminates κ from leading terms in regret.

Improvements by LightONS. By replacing the ONS-based subroutine with a LightONS-based counterpart, we retain the same regret bound and improve the runtime from $\widetilde{O}((d^2K + d^\omega)T)$ to $\widetilde{O}(d^2KT + d^\omega \cdot \min\{\sqrt{\kappa dT}\log\kappa, T\})$, where K is the number of arms for bandits. When the regret is sublinear in T, i.e., $\kappa = o(T)$, our improvement yields a strictly lower runtime order.

OQNS can hardly be adapted to the method of Zhang et al. [2025]. The reasons are two-fold: *(i)* The GLB analysis treats the ONS-based subroutine as a white-box, exploiting negative terms in the OMD analysis, while OQNS fails to provide such analytical properties due to its deviation from OMD; *(ii)* The GLB method relies on non-monotonic local norms, while OQNS is tailored for monotonic local norms as in Eq. (4), to which OQNS's analysis is deeply coupled. On the contrary, the key ingredients of LightONS (deferred projection and domain conversion) are largely orthogonal to the core mirror-descent updates, seamlessly accommodating customized local norms and preserving necessary analytical properties for GLB. Details are deferred to Appendix E.2.

5.3. Memory-Efficient OXO

Another computational challenge of ONS is its $O(d^2)$ working memory, in contrast to OGD's $O(d)$ working memory. Luo et al. [2016] propose Sketched Online Newton Step (SON), which mitigates this issue by incorporating matrix sketching into ONS.

Current results. SON achieves linear-in-d runtime and working memory when the sketched dimension $d' \ll d$. The value of d' typically depends on the problem's intrinsic dimensionality, such as the number of non-zero eigenvalues in the Hessian-related matrix. However, SON demands additional assumptions. Its domain $\mathcal{X}_t$ must be an intersection of two parallel half-spaces, onto which Mahalanobis projections admit closed-form with $O(d^2)$ time. Under the standard OXO setting (Assumptions 1–3), SON loses its computational advantage and reverts to the high computational cost of ONS. The domain restriction further imposes a stronger assumption on loss functions. The curvature parameter $\gamma_0 = \frac{1}{2}\min\{\frac{1}{DG}, \alpha\}$ in Lemma 1 collapses to zero for the unbounded

intersection-of-parallel-half-spaces domain, reducing exp-concavity to convexity. Consequently, SON assumes an explicit quadratic property mirroring Lemma 1, rather than the standard Assumption 3.

Improvements by LightONS. We propose LightONS.Sketch (Algorithm 6 in Appendix E.3), which replaces LightONS's accesses to the Hessian-related matrix A_t with the sketching primitives as in SON. LightONS.Sketch combines the projection efficiency of LightONS with the memory efficiency of SON. This hybrid method extends linear-in-d runtime and working memory to the standard OXO setting, while retaining SON's regret $O(d' \log T)$. Theoretical guarantees of the hybrid method are summarized in the following theorem.

Theorem 4 (LightONS's improvement for memory-efficient OXO) *Under Assumptions 1–3, with* $\gamma_0 = \frac{1}{2}\min\{\frac{1}{DG}, \alpha\}$, *LightONS.Sketch (Algorithm 6) satisfies that, for any* $\mathbf{u} \in \mathcal{X}$,

$$\textsc{Reg}_T(\mathbf{u}) \le \frac{d'}{\gamma_0}\log\left(1 + \frac{G^2}{2d'\epsilon}T\right) + \frac{\gamma_0 \epsilon D^2}{8} + \frac{\Delta_{1:T}}{2\gamma_0}, \tag{15a}$$

$$\textsc{Runtime} \le O\left(\left(\mathsf{EP}_{\mathcal{X}} + d'd\log T\right)T + d^{\omega}\sqrt{(d+\Delta_{1:T})T/\epsilon}\log T\right), \tag{15b}$$

and the working memory is $O(d'd)$. *With the surrogate gradient* $\nabla g_t(\mathbf{y}_t)$ *as in Lemma 3, the cumulative sketching error* $\Delta_{1:T}$ *can be bounded as*

$$\Delta_{1:T} \le \min_{j\in[d']} \frac{2d'}{(d'-j+1)\epsilon}\sum_{i=j}^{d}\lambda_i\left(\sum_{t=1}^{T}\nabla g_t(\mathbf{y}_t)\nabla g_t(\mathbf{y}_t)^{\intercal}\right). \tag{16}$$

Although Mhammedi and Gatmiry [2023] suggest integrating sketching into OQNS, their algorithm's considerable structural complexity may pose significant obstacles to algorithmic design and theoretical analysis. Details are deferred to Appendix E.3.

6. Conclusion

For online exp-concave optimization (OXO), we propose LightONS, a simple yet powerful variant of ONS. By combining the deferred-projection mechanism with the improper-to-proper conversion, LightONS achieves significant reductions in worst-case total runtime while preserving the regret optimality of ONS. These gains extend to the stochastic optimization setting, answering a COLT'13 open problem on efficient and optimal SXO [Koren, 2013]. Moreover, due to its fidelity to the mirror-descent update of ONS, LightONS serves as an efficient drop-in replacement of ONS across diverse applications, including gradient-norm adaptivity, parametric stochastic bandits, and memory-efficient settings, all without compromising statistical guarantees.

Several important directions remain open. First, while LightONS adapts to gradient norms, achieving adaptivity to *gradient variation* for OXO within $\widetilde{O}(d^2T)$ runtime remains an open challenge [Chiang et al., 2012; Zhao et al., 2024]. Second, applying the LightONS technique to other preconditioned online learning algorithms may not improve their (asymptotic) runtime if the bottleneck lies in other operations, such as explicit matrix factorization rather than Mahalanobis projections (e.g., in AdaGrad [Duchi et al., 2011]). Accelerating these algorithms remains unclear. These future directions motivate the development of broader, general-purpose acceleration techniques for Hessian-related online learning algorithms.

Acknowledgments

This work was supported by National Science and Technology Major Project (2022ZD0114800), NSFC (62361146852), the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM118), and the "111 Center" (No. B26023).

References

Jacob Abernethy, Peter L Bartlett, Alexander Rakhlin, and Ambuj Tewari. Optimal strategies and minimax lower bounds for online convex games. In *Proceedings of the 21st Annual Conference on Learning Theory (COLT)*, pages 415–423, 2008.

Josh Alman, Ran Duan, Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. More asymmetry yields faster matrix multiplication. In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2005–2039, 2025.

Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

Sébastien Bubeck, Ronen Eldan, and Joseph Lehec. Sampling from a log-concave distribution with projected langevin monte carlo. *Discrete & Computational Geometry*, 59:757–783, 2018.

T Tony Cai, Cun-Hui Zhang, and Harrison H Zhou. Optimal rates of convergence for covariance matrix estimation. *The Annals of Statistics*, 38(4):2118–2144, 2010.

Nicolò Cesa-Bianchi and Gábor Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, 2006.

Chao-Kai Chiang, Tianbao Yang, Chia-Jung Lee, Mehrdad Mahdavi, Chi-Jen Lu, Rong Jin, and Shenghuo Zhu. Online optimization with gradual variations. In *Proceedings of the 25th Conference On Learning Theory (COLT)*, pages 6.1–6.20, 2012.

Thomas M Cover. Universal portfolios. *Mathematical Finance*, 1(1):1–29, 1991.

Ashok Cutkosky. Parameter-free, dynamic, and strongly-adaptive online learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 2250–2259, 2020.

Ashok Cutkosky and Francesco Orabona. Black-box reductions for parameter-free online learning in Banach spaces. In *Proceedings of the 31st Conference on Learning Theory (COLT)*, pages 1493–1529, 2018.

John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(7):2121–2159, 2011.

Dean P Foster. Prediction in the worst case. *The Annals of Statistics*, pages 1084–1090, 1991.

Dylan J. Foster and Max Simchowitz. Logarithmic regret for adversarial online control. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 3211–3221, 2020.

Dylan J Foster, Satyen Kale, Haipeng Luo, Mehryar Mohri, and Karthik Sridharan. Logistic regression: The importance of being improper. In *Proceedings of 31st Conference on Learning Theory (COLT)*, pages 167–208, 2018.

Dan Garber and Ben Kretzu. Projection-free online exp-concave optimization. In *Proceedings of the 36th Annual Conference on Learning Theory (COLT)*, pages 1259–1284, 2023.

Mina Ghashami, Edo Liberty, Jeff M. Phillips, and David P. Woodruff. Frequent directions: Simple and deterministic matrix sketching. *SIAM Journal on Computing*, 45(5):1762–1792, 2016.

Gene H Golub and Charles F Van Loan. *Matrix Computations*. JHU Press, 2013.

Elad Hazan. Introduction to Online Convex Optimization. *Foundations and Trends® in Optimization*, 2(3-4):157–325, 2016.

Elad Hazan and Satyen Kale. Beyond the regret minimization barrier: An optimal algorithm for stochastic strongly-convex optimization. In *Proceedings of the 24th Annual Conference on Learning Theory (COLT)*, pages 421–436, 2011.

Elad Hazan and Nimrod Megiddo. An efficient interior-point method for online convex optimization. *ArXiv preprint*, arXiv:2307.11668, 2023.

Elad Hazan, Amit Agarwal, and Satyen Kale. Logarithmic regret algorithms for online convex optimization. *Machine Learning*, 69(2-3):169–192, 2007.

Elad Hazan, Tomer Koren, and Kfir Y Levy. Logistic regression: Tight bounds for stochastic and online optimization. In *Proceedings of 27th Conference on Learning Theory (COLT)*, pages 197–209, 2014.

Oscar H Ibarra, Shlomo Moran, and Roger Hui. A generalization of the fast LUP matrix decomposition algorithm and applications. *Journal of Algorithms*, 3(1):45–56, 1982.

Haotian Jiang, Yin Tat Lee, Zhao Song, and Sam Chiu-wai Wong. An improved cutting plane method for convex optimization, convex-concave games, and its applications. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 944–953, 2020.

Kwang-Sung Jun, Aniruddha Bhargava, Robert D. Nowak, and Rebecca Willett. Scalable generalized linear bandits: Online computation and hashing. In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 99–109, 2017.

Jyrki Kivinen and Manfred K Warmuth. Averaging expert predictions. In *Proceedings of 4th European Conference on Computational Learning Theory (EuroCOLT)*, pages 153–167, 1999.

Tomer Koren. Open problem: Fast stochastic exp-concave optimization. In *Proceedings of the 26th Annual Conference on Learning Theory (COLT)*, pages 1073–1075, 2013.

Tomer Koren and Kfir Y. Levy. Fast rates for exp-concave empirical risk minimization. In *Advances in Neural Information Processing Systems 28 (NIPS)*, pages 1477–1485, 2015.

Yin Tat Lee, Aaron Sidford, and Sam Chiu-wai Wong. A faster cutting plane method and its implications for combinatorial and convex optimization. In *Proceedings of the 56th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1049–1065, 2015.

Haipeng Luo, Alekh Agarwal, Nicolò Cesa-Bianchi, and John Langford. Efficient second order online learning by sketching. In *Advances in Neural Information Processing Systems 29 (NIPS)*, pages 902–910, 2016.

Mehrdad Mahdavi, Lijun Zhang, and Rong Jin. Lower and upper bounds on the generalization of stochastic exponentially concave optimization. In *Proceedings of the 28th Conference on Learning Theory (COLT)*, pages 1305–1320, 2015.

Nishant Mehta. Fast rates with high probability in exp-concave statistical learning. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 1085–1093, 2017.

Zakaria Mhammedi and Khashayar Gatmiry. Quasi-newton steps for efficient online exp-concave optimization. In *Proceedings of The 36th Annual Conference on Learning Theory (COLT)*, pages 4473–4503, 2023.

Yurii E. Nesterov and Arkadii Nemirovskii. *Interior-Point Polynomial Algorithms in Convex Programming*. SIAM, 1994.

Francesco Orabona. A Modern Introduction to Online Learning. *ArXiv preprint*, arxiv:1912.13213, 2019.

Francesco Orabona, Nicolò Cesa-Bianchi, and Claudio Gentile. Beyond logarithmic bounds in online learning. In *Proceedings of the 15th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 823–831, 2012.

Erik Ordentlich and Thomas M Cover. The cost of achieving the best portfolio in hindsight. *Mathematics of Operations Research*, 23(4):960–982, 1998.

Beresford N Parlett. *The Symmetric Eigenvalue Problem*. SIAM, 1998.

Kaare Petersen and Michael Petersen. *The Matrix Cookbook*. 2012. URL `https://ece.uwaterloo.ca/~ece602/MISC/matrixcookbook.pdf`.

Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014.

Max Simchowit. Making non-stochastic control (almost) as easy as stochastic. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, pages 18318–18329, 2020.

Nathan Srebro, Karthik Sridharan, and Ambuj Tewari. Smoothness, low noise and fast rates. In *Advances in Neural Information Processing Systems 23 (NIPS)*, pages 2199–2207, 2010.

Dirk van der Hoeven, Nikita Zhivotovskiy, and Nicolò Cesa-Bianchi. High-probability risk bounds via sequential predictors. *ArXiv preprint*, arXiv:2308.07588, 2023.

Volodya Vovk. Competitive on-line linear regression. In *Advances in Neural Information Processing Systems 10 (NIPS)*, pages 364–370, 1997.

Yuanyu Wan, Guanghui Wang, Wei-Wei Tu, and Lijun Zhang. Projection-free distributed online learning with sublinear communication complexity. *Journal of Machine Learning Research*, 23 (172):1–53, 2022.

Wenhao Yang, Yibo Wang, Peng Zhao, and Lijun Zhang. Universal online convex optimization with 1 projection per round. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, pages 31438–31472, 2024.

Lijun Zhang, Tianbao Yang, Rong Jin, Yichi Xiao, and Zhi-Hua Zhou. Online stochastic linear optimization under one-bit feedback. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 392–401, 2016.

Yu-Jie Zhang, Sheng-An Xu, Peng Zhao, and Masashi Sugiyama. Generalized linear bandits: Almost optimal regret with one-pass update. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, pages 69244–69277, 2025.

Peng Zhao, Yu-Jie Zhang, Lijun Zhang, and Zhi-Hua Zhou. Dynamic regret of convex and smooth functions. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, pages 12510–12520, 2020.

Peng Zhao, Yu-Jie Zhang, Lijun Zhang, and Zhi-Hua Zhou. Adaptivity and non-stationarity: Problem-dependent dynamic regret for online convex optimization. *Journal of Machine Learning Research*, 25(98):1–52, 2024.

Peng Zhao, Yan-Feng Xie, Lijun Zhang, and Zhi-Hua Zhou. Efficient methods for non-stationary online learning. *Journal of Machine Learning Research*, 25(208):1–66, 2025.

Martin Zinkevich. Online convex programming and generalized infinitesimal gradient ascent. In *Proceedings of the 20th International Conference on Machine Learning (ICML)*, pages 928–936, 2003.

Appendix A. Empirical Validation

We conduct numerical experiments to validate the theoretical guarantees of LightONS, especially its negligible statistical gap from ONS and its non-asymptotic statistical advantages over OQNS.

Setup. We evaluate on two fundamental tasks, linear and logistic regression, over the Euclidean ball domain $\mathcal{X} = \mathcal{B}(D/2) \subset \mathbb{R}^d$ over a time horizon of T. With $\{(\mathbf{x}_t, y_t)\}_{t=1}^T$ sampled i.i.d. from the folded standard Gaussian distribution, i.e., each entry of $\mathbf{x}_t$ and y_t is the absolute value of a standard Gaussian random variable, the (online) loss functions are designed as

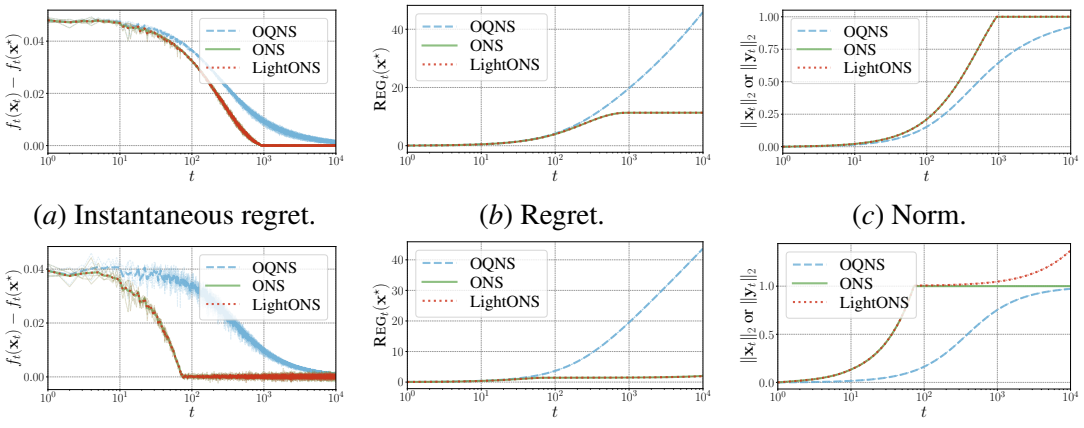
$$\ell_t^{\text{linear}}(\mathbf{w}) = \frac{1}{2}\left(\sqrt{\frac{G}{D}}\mathbf{x}_t^\intercal \mathbf{w} + \frac{\sqrt{DG}}{2} y_t\right)^2, \quad \ell_t^{\text{logistic}}(\mathbf{w}) = \log\left(1 + \exp\left(G\mathbf{x}_t^\intercal \mathbf{w}\right)\right).$$

Both loss functions are G-Lipschitz and α-exp-concave with $\alpha = 1/(DG)$ and $\alpha = \exp(-DG)$, respectively. Our logistic regression setup slightly differs from the standard formulation, as we omit the binary label $y_t \in \{0, 1\}$ for simplicity, which does not affect exp-concavity.

Three algorithms, LightONS, ONS, and OQNS, are configured with their theoretically optimal parameters. The implementation of ONS and LightONS follows Algorithms 1 and 3, respectively; OQNS is implemented in accordance with Algorithm 3 of Mhammedi and Gatmiry [2023].

Results. Figure 1 displays all results averaged over 5 independent runs with the same seeds, where the averaged performance is shown as a dark line, while individual runs are in transparent lines.

Our experiments confirm that computational gains of LightONS incur negligible loss in statistical performance. As shown in Figures 1(*a*), 1(*b*), 1(*d*) and 1(*e*), curves of LightONS (red dotted) overlap with those of ONS (green solid). This provides empirical evidence that LightONS retains the sharp regret constants of ONS. In contrast, OQNS (blue dashed) shows noticeably higher regret, indicating that its theoretical bounds may carry greater constants.



(*a*) Instantaneous regret. (*b*) Regret. (*c*) Norm.

(*d*) Instantaneous regret. (*e*) Regret. (*f*) Norm.

Figure 1: Numerical results with $T = 10^4$, $d = 10$, and $D = 2$. The first row shows linear regression results with $G = 1/10$, $\alpha = 5$ and the second row shows logistic regression results with $G = 1/10$, $\alpha = \exp(-1/5)$.

The key advantage of LightONS, its computational efficiency, is evident in Figures 1(*c*) and 1(*f*). After an initial convergence phase (e.g., the first 10^2 rounds in Figure 1(*f*)), both ONS and LightONS stabilize near the offline optimal solution. Then ONS frequently pushes decisions outside the domain, triggering costly Mahalanobis projections. In contrast, LightONS consistently outputs decisions with $\|\mathbf{y}_t\| \leq kD/2 = 2$ and avoids Mahalanobis projections.

Appendix B. Technical Lemmas

In this section, we present technical lemmas used in the subsequent analysis.

B.1. Proof of Lemma 1

Proof Since $g(\mathbf{x}) = \mathrm{e}^{-\alpha f(\mathbf{x})}$ is concave and positive, we have that $h(\mathbf{x}) = \mathrm{e}^{-2\gamma_0 f(\mathbf{x})}$ is also concave. Specifically, for any $\lambda \in [0,1]$ and any $(\mathbf{x}, \mathbf{y}) \in \mathcal{X}^2$, it holds that

$$\begin{aligned} h(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) &= g(\lambda\mathbf{x} + (1-\lambda)\mathbf{y})^{2\gamma_0/\alpha} \geq (\lambda g(\mathbf{x}) + (1-\lambda)g(\mathbf{y}))^{2\gamma_0/\alpha} \\ &\geq \lambda g(\mathbf{x})^{2\gamma_0/\alpha} + (1-\lambda)g(\mathbf{y})^{2\gamma_0/\alpha} = \lambda h(\mathbf{x}) + (1-\lambda)h(\mathbf{y}), \end{aligned}$$

where the first inequality follows from the concavity of g and the second from the concavity of $a \mapsto a^{2\gamma_0/\alpha}$ (where $2\gamma_0/\alpha \leq 1$). Next, by the concavity of h, for any $(\mathbf{x}, \mathbf{u}) \in \mathcal{X}^2$, we have

$$\mathrm{e}^{-2\gamma_0 f(\mathbf{u})} \leq \mathrm{e}^{-2\gamma_0 f(\mathbf{x})} + \left(-2\gamma_0 \mathrm{e}^{-2\gamma_0 f(\mathbf{x})} \nabla f(\mathbf{x})\right)^{\mathsf{T}} (\mathbf{u} - \mathbf{x}).$$

Rearranging the preceding inequality yields

$$f(\mathbf{x}) - f(\mathbf{u}) \leq \frac{1}{2\gamma_0} \log\left(1 - 2\gamma_0 \nabla f(\mathbf{x})^{\mathsf{T}} (\mathbf{u} - \mathbf{x})\right).$$

To prove Eq. (2), it suffices to apply that $\log(1+a) \leq a - a^2/4$ (which holds for any $|a| \leq 1$) to the right-hand side of the preceding inequality. We note that $\left|2\gamma_0 \nabla f(\mathbf{x})^{\mathsf{T}} (\mathbf{x} - \mathbf{u})\right| \leq 1$ due to the selection of the curvature parameter $\gamma_0 = \frac{1}{2}\min\{\frac{1}{DG}, \alpha\}$. ■

B.2. Elliptical Potential Lemmas

We introduce two lemmas tackling Hessian-related matrices, which are essential for both regret and runtime analysis. Lemma 5 is standard in the ONS literature (e.g., [Hazan et al., 2007; Luo et al., 2016]). The proof of Lemma 6 mirrors that of Lemma 5, differing only in the potential function: $F(A) = -\operatorname{tr}(A^{-1})$ versus $F(A) = \log\det(A)$.

Lemma 5 *Let $A_i = \lambda I + \sum_{j=1}^{i} \mathbf{v}_j \mathbf{v}_j^{\mathsf{T}}$, if $\|\mathbf{v}_i\|_2 \leq L$ for any $i \in [n]$, then*

$$\sum_{i=1}^{n} \mathbf{v}_i^{\mathsf{T}} A_i^{-1} \mathbf{v}_i \leq \log\det(A_n) - \log\det(A_0) \leq d \log\left(1 + \frac{L^2}{d\lambda} n\right). \tag{17}$$

Lemma 6 *Let $A_i = \lambda I + \sum_{j=1}^{i} \mathbf{v}_j \mathbf{v}_j^{\mathsf{T}}$, then*

$$\sum_{i=1}^{n} \mathbf{v}_i^{\mathsf{T}} A_i^{-2} \mathbf{v}_i \leq \operatorname{tr}(A_0^{-1}) - \operatorname{tr}(A_n^{-1}) \leq \frac{d}{\lambda}. \tag{18}$$

Algorithm 4 FastProj onto Euclidean ball $\mathcal{B}(R)$ with Mahalanobis norm $\|\cdot\|_A$

Input: point $\mathbf{y} \notin \mathcal{B}(R)$, error tolerance ζ, range of A's eigenvalues $[\underline{\lambda}, \overline{\lambda}]$.
Output: approximate Mahalanobis projection $\mathbf{x} \approx \Pi^A_{\mathcal{B}(R)}[\mathbf{y}]$.

1: (Choice 1.) $\mathbf{v} = A\mathbf{y}$, let $\rho(\mu) = \left\| (A + \mu I)^{-1} \mathbf{v} \right\|_2^2 - R^2$.
2: (Choice 2.) Tridiagonalize $QCQ^\mathsf{T} = A$, $\mathbf{w} = CQ^\mathsf{T}\mathbf{y}$, let $\rho(\mu) = \left\| (C + \mu I)^{-1} \mathbf{w} \right\|_2^2 - R^2$.
3: $a_1 = (\frac{\|\mathbf{y}\|_2}{R} - 1)\underline{\lambda}$, $b_1 = (\frac{\|\mathbf{y}\|_2}{R} - 1)\overline{\lambda}$, $n = \left\lceil \log_2(\frac{1}{\zeta}(\frac{\overline{\lambda}}{\underline{\lambda}} - 1) \|\mathbf{y}\|_2 (\frac{\|\mathbf{y}\|_2}{R} - 1)) \right\rceil$.
(Without loss of generality, we assume $n \geq 1$.)
4: **for** $i = 1, \ldots, n$ **do**
5: $(a_{i+1}, b_{i+1}) = \begin{cases} (\frac{a_i + b_i}{2}, b_i), & \text{if } \rho(\frac{a_i + b_i}{2}) \geq 0; \\ (a_i, \frac{a_i + b_i}{2}), & \text{otherwise.} \end{cases}$
6: **end for**
7: $\mathbf{x} = \min\{1, \frac{R}{\|\widetilde{\mathbf{x}}\|_2}\} \cdot \widetilde{\mathbf{x}}$, where $\widetilde{\mathbf{x}} = (A + \frac{a_{n+1} + b_{n+1}}{2} I)^{-1} A\mathbf{y}$.

Proof of Lemmas 5 and 6 Let $\mathbb{S}_{++}$ denote the set of d-dimensional positive-definite and symmetric matrices, and let $\langle \cdot, \cdot \rangle_{\mathrm{F}}$ denote the inner product induced by the Frobenius matrix norm. Then for any concave function $F : \mathbb{S}_{++} \to \mathbb{R}$, we have

$$\sum_{i=1}^{n} \mathbf{v}_i^\mathsf{T} \nabla F(A_i) \mathbf{v}_i = \sum_{i=1}^{n} \langle \nabla F(A_i), A_i - A_{i-1} \rangle_{\mathrm{F}} \leq \sum_{i=1}^{n} (F(A_i) - F(A_{i-1})) = F(A_n) - F(A_0). \tag{19}$$

The first equality follows from $A_i - A_{i-1} = \mathbf{v}_i \mathbf{v}_i^\mathsf{T}$ (by the definition of A_i) and $\mathbf{u}^\mathsf{T} A \mathbf{v} = \left\langle A, \mathbf{v}\mathbf{u}^\mathsf{T} \right\rangle_{\mathrm{F}}$ (by the definition of the $\langle \cdot, \cdot \rangle_{\mathrm{F}}$); The second inequality from the concavity of F; The last equality from telescoping. Substituting the respective potentials into Eq. (19) yields Lemmas 5 and 6. Concretely,

- For Lemma 5, $F(A) = \log\det(A)$ and $\nabla F(A) = A^{-1}$.
- For Lemma 6, $F(A) = -\operatorname{tr}(A^{-1})$ and $\nabla F(A) = A^{-2}$.

Concavity of F is standard [Boyd and Vandenberghe, 2004] and gradients of F follow from matrix calculus [Petersen and Petersen, 2012]. ■

B.3. Numerical Lemmas

The overall efficiency of LightONS hinges on two key operations, namely the matrix inversion A_t^{-1} and the (infrequent) Mahalanobis projection $\Pi^{A_t}_{\mathcal{B}(D/2)}[\widehat{\mathbf{y}}_{t+1}]$. In this subsection, we detail efficient numerical approaches for the two operations, respectively.

Matrix inversion. The per-round update of the matrix A_t is a rank-one update. Instead of inverting from scratch at a cost of $O(d^\omega)$, we can update A_{t-1}^{-1} to obtain A_t^{-1} in only $O(d^2)$ time with the Sherman-Morrison-Woodbury formula. The following equation ensures $V_t = A_t^{-1}$ for any $t \in [T]$.

$$V_0 = \frac{1}{\epsilon} I; \quad V_t = V_{t-1} - \frac{1}{1 + \|\boldsymbol{\nabla}_t\|^2_{V_{t-1}}} V_{t-1} \boldsymbol{\nabla}_t \boldsymbol{\nabla}_t^\mathsf{T} V_{t-1}, \quad \text{where } \boldsymbol{\nabla}_t = c_f \nabla g_t(\mathbf{y}_t). \tag{20}$$

Mahalanobis projection onto Euclidean ball. In LightONS, all Mahalanobis projections are onto the Euclidean ball $\mathcal{B}(D/2)$ rather than the potentially complex domain $\mathcal{X}$. This geometry enables customized numerical approaches faster than generic solvers [Lee et al., 2015; Jiang et al., 2020]. Specifically, the dual problem of the Mahalanobis projection reduces to a one-dimensional root-finding problem (Exercise 4.22 of Boyd and Vandenberghe [2004]), solvable via bisection.

We propose FastProj in Algorithm 4, which offers two implementation choices for different purposes. Choice 1 attains superior theoretical dependence on d by exploiting fast matrix multiplication with exponent $\omega < 2.3714$ [Alman et al., 2025]. Choice 2 offers stronger practical performance via tridiagonalization tricks [Parlett, 1998; Golub and Van Loan, 2013], which are more favorable in practice where $\omega = 3$. The following lemma characterizes the error and runtime of FastProj.

Lemma 7 *Let $\mathbf{x}$ be the output of FastProj (Algorithm 4), and $\mathbf{x}^* = \Pi^A_{\mathcal{B}(R)}[\mathbf{y}]$ be the exact Mahalanobis projection, then $\mathbf{x} \in \mathcal{B}(R)$ and $\|\mathbf{x} - \mathbf{x}^*\|_2 \le \zeta$. The number of bisections is $O(\log(\frac{1}{\zeta}\frac{\overline{\lambda}}{\underline{\lambda}}))$. With choice 1, the runtime is $O(d^\omega n)$, and with choice 2, the runtime is $O(d^3 + dn)$.*

In the context of LightONS (Algorithm 3), it suffices to set the error tolerance as $\zeta_t = O(1/t^2)$ at the t-th round. The accumulated truncation error contributes only a negligible additive constant to the regret of LightONS. A detailed discussion is deferred to Lemma 12 in Appendix C.4. Moreover, with $\zeta_t = \Omega(1/t^3)$, $\underline{\lambda}_t = \epsilon$ and $\overline{\lambda}_t = \epsilon + c_f^2 c_g^2 G^2 t$, we have $n_t = O(\log t)$ at the t-th round.

To prove Lemma 7, we first present two supporting lemmas. Lemma 8 reduces the Mahalanobis projection onto a Euclidean ball to a one-dimensional root-finding problem. Lemma 9 bridges the truncation error of root-finding and the truncation error of Mahalanobis projection.

Lemma 8 $\Pi^A_{\mathcal{B}(R)}[\mathbf{y}] = (A + \mu^* I)^{-1} A\mathbf{y}$, *where μ^* is the only positive zero of the following function*

$$\rho(\mu) = \left\| (A + \mu I)^{-1} A\mathbf{y} \right\|_2^2 - R^2 = \sum_{i=1}^{d} \frac{v_i^2}{\left(1 + \frac{\mu}{\lambda_i}\right)^2} - R^2, \tag{21}$$

and μ^* satisfies that

$$\left(\frac{\|\mathbf{y}\|_2}{R} - 1\right)\lambda_d \le \mu^* \le \left(\frac{\|\mathbf{y}\|_2}{R} - 1\right)\lambda_1, \tag{22}$$

where $v_i = \mathbf{e}_i^\mathsf{T} Q^\mathsf{T} \mathbf{y}$, $\mathbf{e}_i$ is the i-th standard basis vector, and $A = Q\Lambda Q^\mathsf{T}$ is the eigendecomposition, $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_d)$ with $\lambda_1 \ge \cdots \ge \lambda_d > 0$.

Proof of Lemma 8 The Mahalanobis projection onto a Euclidean ball is formulated as

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^d} \ & (\mathbf{x} - \mathbf{y})^\mathsf{T} A (\mathbf{x} - \mathbf{y}) \\ \text{s.t.} \ & \mathbf{x}^\mathsf{T}\mathbf{x} \le R^2 \end{aligned}$$

The Lagrangian of this quadratic program is $\mathcal{L} = (\mathbf{x} - \mathbf{y})^\mathsf{T} A (\mathbf{x} - \mathbf{y}) + \mu(\mathbf{x}^\mathsf{T}\mathbf{x} - R^2)$. According to the KKT conditions, $\nabla\mathcal{L} = 2A(\mathbf{x} - \mathbf{y}) + 2\mu\mathbf{x} = \mathbf{0}$ with $\mu > 0$. (Otherwise, $\mu = 0$ implies $\mathbf{y} \in \mathcal{B}(R)$ and the projection is trivial.) Rearranging $\nabla\mathcal{L} = \mathbf{0}$ yields $\mathbf{x} = (A + \mu I)^{-1} A\mathbf{y}$, thus $\mathbf{x}^\mathsf{T}\mathbf{x} = R^2$ is equivalent to $\rho(\mu) = 0$, which proves Eq. (21).

To prove Eq. (22), since ρ monotonically decreases on $[0,\infty)$, it suffices to bound $\rho(\mu)$. By the orthogonality of Q, we have $\sum_{i=1}^d v_i^2 = \left\|Q^\mathsf{T}\mathbf{y}\right\|_2^2 = \|\mathbf{y}\|_2^2$. Therefore,

$$\frac{\|\mathbf{y}\|_2^2}{(1+\frac{\mu}{\lambda_d})^2} - R^2 \le \rho(\mu) = \sum_{i=1}^d \frac{v_i^2}{(1+\frac{\mu}{\lambda_i})^2} - R^2 \le \frac{\|\mathbf{y}\|_2^2}{(1+\frac{\mu}{\lambda_1})^2} - R^2.$$

Then it is straightforward that $\rho((\frac{\|\mathbf{y}\|_2}{R}-1)\lambda_d) \ge 0$ and $\rho((\frac{\|\mathbf{y}\|_2}{R}-1)\lambda_1) \le 0$, yielding that $\mu^* \in [(\frac{\|\mathbf{y}\|_2}{R}-1)\lambda_d, (\frac{\|\mathbf{y}\|_2}{R}-1)\lambda_1]$. Since ρ is strictly decreasing on $[0,\infty)$ and μ^* is its unique positive zero. ■

Lemma 9 *Let $\mathbf{x}^* = (A+\mu^* I)^{-1}A\mathbf{y}$, where $\rho(\mu^*) = 0$ as in Lemma 8. If $\widetilde{\mu} \ge 0$ and $|\widetilde{\mu}-\mu^*| \le \frac{\lambda_d}{\|\mathbf{y}\|_2}\zeta$, then $\mathbf{x} \in \mathcal{B}(R)$ and $\|\mathbf{x}-\mathbf{x}^*\|_2 \le \zeta$, where $\mathbf{x}$ is constructed from $\widetilde{\mu}$ as*

$$\mathbf{x} = \Pi_{\mathcal{B}(R)}[\widetilde{\mathbf{x}}] = \min\left\{1, \frac{R}{\|\widetilde{\mathbf{x}}\|_2}\right\}\cdot\widetilde{\mathbf{x}}, \quad \widetilde{\mathbf{x}} = (A+\widetilde{\mu}I)^{-1}A\mathbf{y}. \tag{23}$$

Proof of Lemma 9 Eq. (23) immediately implies $\mathbf{x} \in \mathcal{B}(R)$.

To prove $\|\mathbf{x}-\mathbf{x}^*\|_2 \le \zeta$, since $\|\mathbf{x}-\mathbf{x}^*\|_2 \le \|\widetilde{\mathbf{x}}-\mathbf{x}^*\|_2$ by Lemma 10, it suffices to prove $\|\widetilde{\mathbf{x}}-\mathbf{x}^*\|_2 \le \zeta$. By the definition of v_i in Lemma 8, we have

$$\begin{aligned}\|\widetilde{\mathbf{x}}-\mathbf{x}^*\|_2^2 &= \left\|(A+\widetilde{\mu}I)^{-1}A\mathbf{y} - (A+\mu^*I)^{-1}A\mathbf{y}\right\|_2^2 = \sum_{i=1}^d v_i^2\left(\frac{1}{1+\frac{\widetilde{\mu}}{\lambda_i}} - \frac{1}{1+\frac{\mu^*}{\lambda_i}}\right)^2 \\ &\le \sum_{i=1}^d v_i^2\left(1-\frac{1}{1+\frac{|\widetilde{\mu}-\mu^*|}{\lambda_i}}\right)^2 \le \sum_{i=1}^d v_i^2\left(\frac{\widetilde{\mu}-\mu^*}{\lambda_i}\right)^2 \le \|\mathbf{y}\|_2^2\left(\frac{\widetilde{\mu}-\mu^*}{\lambda_d}\right)^2 \le \zeta^2.\end{aligned}$$

The first inequality uses that $\lambda_i > 0$, $\widetilde{\mu} \ge 0$ and $\mu^* \ge 0$; The second inequality comes from the following inequality, $(1-\frac{1}{1+a})^2 = \frac{a^2}{(a+1)^2} \le a^2$ for any $a \ge 0$; The third inequality uses that $\sum_{i=1}^d v_i^2 = \|\mathbf{y}\|_2^2$; The last inequality uses that $|\widetilde{\mu}-\mu^*| \le \frac{\lambda_d}{\|\mathbf{y}\|_2}\zeta$. ■

With the help of these lemmas, we can prove Lemma 7.

Proof of Lemma 7 First, by Lemma 8 and the supplied eigenvalue bounds, the wanted zero satisfies

$$a_1 \le \left(\frac{\|\mathbf{y}\|_2}{R}-1\right)\lambda_d \le \mu^* \le \left(\frac{\|\mathbf{y}\|_2}{R}-1\right)\lambda_1 \le b_1,$$

which justifies the initial bisection interval (Line 3 in Algorithm 4). Furthermore, ρ monotonically decreases on $[0,\infty)$, which implies the selection of a_{i+1} and b_{i+1} (Lines 4–6 in Algorithm 4).

Next, we show the convergence of the bisection. With the interval length halving each iteration, let $\widetilde{\mu} = (a_{n+1}+b_{n+1})/2$. Based on the value of n (Line 3 in Algorithm 4), we have

$$|\widetilde{\mu}-\mu^*| \le \frac{b_1-a_1}{2^n} = \frac{(\frac{\|\mathbf{y}\|_2}{R}-1)(\overline{\lambda}-\underline{\lambda})}{2^n} \le \frac{\underline{\lambda}}{\|\mathbf{y}\|_2}\zeta \le \frac{\lambda_d}{\|\mathbf{y}\|_2}\zeta.$$

Then by Lemma 9, Algorithm 4 achieves an error $\|\mathbf{x}-\mathbf{x}^*\|_2 \le \zeta$.

Finally, we bound the runtime. We note that choice 1 and 2 are equivalent, since

$$\left\|(A+\mu I)^{-1}A\mathbf{y}\right\|_2^2 = \left\|Q\left(C+\mu I\right)^{-1}Q^{\mathsf{T}}QCQ^{\mathsf{T}}\mathbf{y}\right\|_2^2 = \left\|(C+\mu I)^{-1}\mathbf{w}\right\|_2^2.$$

With choice 1, each iteration requires d^ω arithmetic operations, where the computational bottleneck lies in the matrix inversion $(A+\mu I)^{-1}$, resulting in a runtime of $O(d^\omega n)$. With choice 2, the tridiagonalization of A requires $O(d^3)$ arithmetic operations [Golub and Van Loan, 2013], and each iteration can evaluate $(C+\mu I)^{-1}\mathbf{w}$ via the Thomas algorithm with only $O(d)$ arithmetic operations [Golub and Van Loan, 2013], resulting in a runtime of $O(d^3+dn)$. ■

Appendix C. Proofs for Section 3

In this section, we prove the theoretical guarantees of LightONS.

C.1. Proof of Lemma 2

Proof Let Φ_T denote the sum of squared norms of updates, as in the left-hand side of Eq. (18) in Lemma 6. Note that $A_t = \epsilon I + \sum_{i=1}^t \nabla f_t(\mathbf{x}_t)\nabla f_t(\mathbf{x}_t)^{\mathsf{T}}$ in Algorithm 2 and $\|\nabla f_t(\mathbf{x}_t)\|_2 \le G$ in Assumption 2. Then by Lemma 6 we have

$$\Phi_T \triangleq \sum_{t=1}^T \left\|\frac{1}{\gamma}A_t^{-1}\nabla f_t(\mathbf{x}_t)\right\|_2^2 \le \frac{1}{\gamma^2}\frac{d}{\epsilon}.$$

We prove Lemma 2 by bounding N with Φ_T. Let $\{\tau_i\}_{i=1}^N \subseteq \{2,\dots,T+1\}$ denote the indices of all decisions produced by a Mahalanobis projection. With $\tau_0 = 1$, we have that for any $i \in [N]$,

$$-\sum_{t=\tau_{i-1}}^{\tau_i-1}\frac{1}{\gamma}A_t^{-1}\nabla f_t(\mathbf{x}_t) = \widehat{\mathbf{x}}_{\tau_i} - \mathbf{x}_{\tau_{i-1}}.$$

Since $\|\widehat{\mathbf{x}}_{\tau_i}\|_2 > kD/2$ and $\left\|\mathbf{x}_{\tau_{i-1}}\right\|_2 \le D/2$ (because $\mathbf{x}_{\tau_0} = \mathbf{x}_1 = \mathbf{0}$ and every later $\mathbf{x}_{\tau_{i-1}}$ is a projection onto $\mathcal{X}$), we have

$$\left\|\sum_{t=\tau_{i-1}}^{\tau_i-1}\frac{1}{\gamma}A_t^{-1}\nabla f_t(\mathbf{x}_t)\right\|_2 = \left\|\widehat{\mathbf{x}}_{\tau_i} - \mathbf{x}_{\tau_{i-1}}\right\|_2 > (k-1)\frac{D}{2}.$$

By applying the inequality $n\sum_{i=1}^n \|\mathbf{v}_i\|_2^2 \ge \|\sum_{i=1}^n \mathbf{v}_i\|_2^2$, we obtain

$$(\tau_i - \tau_{i-1})\sum_{t=\tau_{i-1}}^{\tau_i-1}\left\|\frac{1}{\gamma}A_t^{-1}\nabla f_t(\mathbf{x}_t)\right\|_2^2 \ge \left\|\sum_{t=\tau_{i-1}}^{\tau_i-1}\frac{1}{\gamma}A_t^{-1}\nabla f_t(\mathbf{x}_t)\right\|_2^2 > (k-1)^2\frac{D^2}{4}.$$

Rearranging the preceding inequality yields

$$\frac{1}{\tau_i - \tau_{i-1}} < \frac{4}{(k-1)^2D^2}\sum_{t=\tau_{i-1}}^{\tau_i-1}\left\|\frac{1}{\gamma}A_t^{-1}\nabla f_t(\mathbf{x}_t)\right\|_2^2.$$

Since $\tau_i - \tau_{i-1} \geq 1$, applying the "harmonic mean $\leq$ arithmetic mean" inequality $\frac{n}{\sum_{i=1}^n \frac{1}{a_i}} \leq \frac{\sum_{i=1}^n a_i}{n}$ for $a_i = \frac{1}{\tau_i - \tau_{i-1}} > 0$, we have

$$\frac{N^2}{T} \leq \frac{N^2}{\tau_N - \tau_0} \leq \sum_{i=1}^{N} \frac{1}{\tau_i - \tau_{i-1}}.$$

Combining the preceding two inequalities, we obtain

$$\frac{N^2}{T} \leq \sum_{i=1}^{N} \frac{1}{\tau_i - \tau_{i-1}} < \frac{4}{(k-1)^2 D^2} \sum_{t=1}^{\tau_N - 1} \left\| \frac{1}{\gamma} A_t^{-1} \nabla f_t(\mathbf{x}_t) \right\|_2^2 \leq \frac{4}{(k-1)^2 D^2} \Phi_T.$$

Finally, rearranging the preceding inequality yields the desired result

$$N < \frac{2\sqrt{T\Phi_T}}{(k-1)D} \leq \frac{2}{(k-1)D\gamma} \sqrt{\frac{d}{\epsilon} T}.$$

■

We remark that Lemma 2 is tight in terms of T up to logarithmic factors. Consider $d = 1$ and $\nabla f_t(\mathbf{x}_t) = 1/\sqrt{T}$, then $A_t^{-1} \nabla f_t(\mathbf{x}_t) = \Omega(1/\sqrt{T})$ and $\| \sum_{t=1}^{T} A_t^{-1} \nabla f_t(\mathbf{x}_t) \|_2 = \Omega(\sqrt{T})$. We notice that Theorem 2.3 of Simchowit [2020] corroborates this point.

C.2. Proof of Theorem 1

Proof The proof of Theorem 1 follows as a specialization of the proof of Theorem 2 given in Appendix C.4. The only substantive difference is that, in the present setting, the analysis is carried out directly on the original loss functions f_t, rather than on surrogate losses g_t.

We therefore omit the repetitive details and highlight only the key intermediate arguments.

- **Choice of the expanded curvature parameter γ in LightONS.Core.** The selection of $\gamma = \frac{1}{2} \min\{\frac{2}{(k+1)DG}, \alpha\}$ directly follows from Lemma 1.
- **Regret analysis with selective projections.** The regret analysis follows the same structure as Lemma 11, which accounts for projections triggered only when the iterate exits the expanded domain $\widetilde{\mathcal{X}}_k$. Details are given in Appendix C.4.
- **Runtime analysis with selective projections.** The runtime bound is derived in the same manner as in the proof of Theorem 2 in Appendix C.4.

■

C.3. Proof of Lemma 4

Proof By the selection of curvature parameters γ_0 (in Lemma 1) and γ' (in Algorithm 3), it holds that $\gamma' = \frac{1}{2} \min\{\frac{1}{DG}, \alpha, \frac{4}{(k+1)c_f c_g DG}\} = \min\{\gamma_0, \frac{2}{(k+1)c_f c_g DG}\} \leq \gamma_0$. Thus by Lemma 1 we have

$$\begin{aligned} f_t(\mathbf{x}_t) - f_t(\mathbf{u}) &\leq \nabla f_t(\mathbf{x}_t)^\mathsf{T}(\mathbf{x}_t - \mathbf{u}) - \frac{\gamma_0}{2} \left(\nabla f_t(\mathbf{x}_t)^\mathsf{T}(\mathbf{x}_t - \mathbf{u}) \right)^2 \\ &\leq \nabla f_t(\mathbf{x}_t)^\mathsf{T}(\mathbf{x}_t - \mathbf{u}) - \frac{\gamma'}{2} \left(\nabla f_t(\mathbf{x}_t)^\mathsf{T}(\mathbf{x}_t - \mathbf{u}) \right)^2. \end{aligned}$$

To finish the proof, it suffices to show that, with $U(a) = a - \frac{\gamma'}{2}a^2$,

$$U\left(\nabla f_t(\mathbf{x}_t)^\mathsf{T}(\mathbf{x}_t - \mathbf{u})\right) \leq U\left(c_f \nabla g_t(\mathbf{y}_t)^\mathsf{T}(\mathbf{y}_t - \mathbf{u})\right).$$

Note that $U'(a) = 1 - \gamma' a$, thus U monotonically increases on $(-\infty, \frac{1}{\gamma'}]$. It can be verified that

$$\nabla f_t(\mathbf{x}_t)^\mathsf{T}(\mathbf{x}_t - \mathbf{u}) \leq c_f \nabla g_t(\mathbf{y}_t)^\mathsf{T}(\mathbf{y}_t - \mathbf{u}) \leq c_f c_g G \cdot \frac{k+1}{2} D \leq \frac{1}{\gamma'}.$$

The first inequality follows from Condition 1; The second inequality from the Cauchy-Schwarz inequality $\mathbf{u}^\mathsf{T}\mathbf{v} \leq \|\mathbf{u}\|_2 \|\mathbf{v}\|_2$; The last inequality from the selection of γ'. ■

C.4. Proof of Theorem 2

We first introduce a property of the Mahalanobis projection. This lemma is often referred to as the Pythagorean theorem in Banach space or the non-expansiveness of projections.

Lemma 10 (Lemma 8 of Hazan et al. [2007]) *If $A \in \mathbb{R}^{d\times d}$ is a positive-definite and symmetric matrix, then for any convex and compact domain $\mathcal{X} \subseteq \mathbb{R}^d$, any point $\mathbf{y} \in \mathbb{R}^d$ and any point $\mathbf{u} \in \mathcal{X}$, $\left\|\Pi_{\mathcal{X}}^A[\mathbf{y}] - \mathbf{u}\right\|_A \leq \|\mathbf{y} - \mathbf{u}\|_A$.*

Based on Lemma 10, we can decompose the regret of LightONS as the following lemma shows.

Lemma 11 *Ignoring the truncation error of Mahalanobis projections, under the same assumptions as Theorem 2, in Algorithm 3, for any $t \in [T]$ and all $\mathbf{u} \in \mathcal{X}$, it holds that, with $\boldsymbol{\nabla}_t = c_f \nabla g_t(\mathbf{y}_t)$,*

$$2\boldsymbol{\nabla}_t^\mathsf{T}(\mathbf{y}_t - \mathbf{u}) \leq \frac{1}{\gamma'} \|\boldsymbol{\nabla}_t\|_{A_t^{-1}}^2 + \gamma' \|\mathbf{y}_t - \mathbf{u}\|_{A_t}^2 - \gamma' \|\mathbf{y}_{t+1} - \mathbf{u}\|_{A_t}^2. \tag{24}$$

Proof of Lemma 11 When $\|\widehat{\mathbf{y}}_{t+1}\|_2 \leq kD/2$, no Mahalanobis projection is triggered, i.e., $\mathbf{y}_{t+1} = \widehat{\mathbf{y}}_{t+1} = \mathbf{y}_t - \frac{1}{\gamma'} A_t^{-1} \boldsymbol{\nabla}_t$, then for any $\mathbf{u} \in \mathbb{R}^d$, we have

$$\|\mathbf{y}_{t+1} - \mathbf{u}\|_{A_t}^2 = \left\|\mathbf{y}_t - \frac{1}{\gamma'} A_t^{-1} \boldsymbol{\nabla}_t - \mathbf{u}\right\|_{A_t}^2.$$

Otherwise, when $\|\widehat{\mathbf{y}}_{t+1}\|_2 > kD/2$ and the Mahalanobis projection is triggered, by Lemma 10, for any $\mathbf{u} \in \mathcal{X}$, we have

$$\|\mathbf{y}_{t+1} - \mathbf{u}\|_{A_t}^2 = \left\|\Pi_{\mathcal{B}(D/2)}^{A_t}\left[\mathbf{y}_t - \frac{1}{\gamma'} A_t^{-1} \boldsymbol{\nabla}_t\right] - \mathbf{u}\right\|_{A_t}^2 \leq \left\|\mathbf{y}_t - \frac{1}{\gamma'} A_t^{-1} \boldsymbol{\nabla}_t - \mathbf{u}\right\|_{A_t}^2.$$

Combining both cases, we conclude that, for any $t \in [T]$ and any $\mathbf{u} \in \mathcal{X}$,

$$\|\mathbf{y}_{t+1} - \mathbf{u}\|_{A_t}^2 \leq \left\|(\mathbf{y}_t - \mathbf{u}) - \frac{1}{\gamma'} A_t^{-1} \boldsymbol{\nabla}_t\right\|_{A_t}^2.$$

Rearranging the preceding inequality yields the desired result of Eq. (24). ■

We note that Lemma 11 ignores the truncation error of the Mahalanobis projections. The matrix factorization underlying Mahalanobis projections is related to eigendecomposition, equivalent to finding roots of a d-degree polynomial, which is not exactly solvable by finitely many arithmetic operations when $d \geq 5$. The next lemma complements the analysis. Thanks to FastProj (Algorithm 4 in Appendix B.3), the truncation error only incurs a negligible additive $O(1/t^2)$ term in the regret decomposition of Lemma 11, which can be safely ignored in the final regret bound of LightONS.

Lemma 12 *Let the Mahalanobis projection of Algorithm 3 be implemented with Algorithm 4 with* $A = A_t$, $R = D/2$, $\mathbf{y} = \widehat{\mathbf{y}}_{t+1}$, $\zeta = \zeta_t$, $\underline{\lambda} = \epsilon$, $\overline{\lambda} = \epsilon + c_f^2 c_g^2 G^2 t$, *where* ζ_t *is defined as*

$$\zeta_t = \min\left\{\frac{1}{(k+1)D \cdot \overline{\lambda} \cdot \gamma' t^2}, \sqrt{\frac{1}{\overline{\lambda} \cdot \gamma' t^2}}\right\} = \Omega\left(\frac{1}{t^3}\right). \tag{25}$$

Then under the same assumptions as Theorem 2, in Algorithm 3, for any $t \in [T]$ *and all* $\mathbf{u} \in \mathcal{X}$, *with* $\boldsymbol{\nabla}_t = c_f \nabla g_t(\mathbf{y}_t)$,

$$2\boldsymbol{\nabla}_t^\mathsf{T}(\mathbf{y}_t - \mathbf{u}) \leq \frac{1}{\gamma'}\left\|\boldsymbol{\nabla}_t\right\|_{A_t^{-1}}^2 + \gamma'\left\|\mathbf{y}_t - \mathbf{u}\right\|_{A_t}^2 - \gamma'\left\|\mathbf{y}_{t+1} - \mathbf{u}\right\|_{A_t}^2 + \frac{2}{t^2}. \tag{26}$$

Proof of Lemma 12 It suffices to consider the case when the Mahalanobis projection is triggered, i.e., $\|\widehat{\mathbf{y}}_{t+1}\|_2 > kD/2$. Let $\mathbf{y}_{t+1}^* = \Pi_{\mathcal{B}(D/2)}^{A_t}[\widehat{\mathbf{y}}_{t+1}]$ be the exact Mahalanobis projection without truncation error, and $\boldsymbol{\delta}_t = \mathbf{y}_{t+1} - \mathbf{y}_{t+1}^*$ be the truncation error. By Lemma 9, $\|\boldsymbol{\delta}_t\|_2 = \left\|\mathbf{y}_{t+1} - \mathbf{y}_{t+1}^*\right\|_2 \leq \zeta_t$; By Lemma 10, for any $\mathbf{u} \in \mathcal{X}$,

$$\begin{aligned}
\left\|\mathbf{y}_{t+1} - \mathbf{u}\right\|_{A_t}^2 &= \left\|\mathbf{y}_{t+1}^* - \mathbf{u}\right\|_{A_t}^2 + 2\left(\mathbf{y}_{t+1}^* - \mathbf{u}\right)^\mathsf{T} A_t \boldsymbol{\delta}_t + \left\|\boldsymbol{\delta}_t\right\|_{A_t}^2 \\
&\leq \left\|\mathbf{y}_{t+1}^* - \mathbf{u}\right\|_{A_t}^2 + (k+1)D \cdot \overline{\lambda} \cdot \left\|\boldsymbol{\delta}_t\right\|_2 + \overline{\lambda} \cdot \left\|\boldsymbol{\delta}_t\right\|_2^2 \\
&\leq \left\|\mathbf{y}_t - \frac{1}{\gamma'} A_t^{-1} \boldsymbol{\nabla}_t - \mathbf{u}\right\|_{A_t}^2 + \frac{2}{\gamma' t^2}.
\end{aligned}$$

The first inequality is because the operator norm of A_t is at most $\overline{\lambda} = c_f^2 c_g^2 G^2 t + \epsilon$, which comes from Condition 1, i.e., $\|\boldsymbol{\nabla}_t\|_2 = \|c_f \nabla g_t(\mathbf{y}_t)\|_2 \leq c_f c_g G$; The second inequality uses the selection of ζ_t in Eq. (25). ■

For conciseness, we ignore the truncation error of the Mahalanobis projections in the proof of Theorem 2, since it only incurs an additive $O(\sum_{t=1}^T 1/t^2) = O(1)$ term in the final regret bound.

Proof of Theorem 2 Recall that $\boldsymbol{\nabla}_t = c_f \nabla g_t(\mathbf{y}_t)$. Plugging Lemma 11 into Lemma 4 yields:

$$\begin{aligned}
f_t(\mathbf{x}_t) - f_t(\mathbf{u}) &\overset{(9)}{\leq} \boldsymbol{\nabla}_t^\mathsf{T}(\mathbf{y}_t - \mathbf{u}) - \frac{\gamma'}{2}\left(\boldsymbol{\nabla}_t^\mathsf{T}(\mathbf{y}_t - \mathbf{u})\right)^2 \\
&\overset{(24)}{\leq} \frac{1}{2}\left(\frac{1}{\gamma'}\left\|\boldsymbol{\nabla}_t\right\|_{A_t^{-1}}^2 + \gamma'\left\|\mathbf{y}_t - \mathbf{u}\right\|_{A_t}^2 - \gamma'\left\|\mathbf{y}_{t+1} - \mathbf{u}\right\|_{A_t}^2\right) - \frac{\gamma'}{2}\left(\boldsymbol{\nabla}_t^\mathsf{T}(\mathbf{y}_t - \mathbf{u})\right)^2 \\
&= \frac{1}{2\gamma'}\left\|\boldsymbol{\nabla}_t\right\|_{A_t^{-1}}^2 + \frac{\gamma'}{2}\left\|\mathbf{y}_t - \mathbf{u}\right\|_{A_{t-1}}^2 - \frac{\gamma'}{2}\left\|\mathbf{y}_{t+1} - \mathbf{u}\right\|_{A_t}^2.
\end{aligned}$$

The equality is because $A_t = A_{t-1} + \boldsymbol{\nabla}_t \boldsymbol{\nabla}_t^\mathsf{T}$ in Algorithm 3. Summing over the time horizon and telescoping the right-hand side establishes the desired regret of Eq. (10a):

$$\begin{aligned}\sum_{t=1}^{T} \left(f_t(\mathbf{x}_t) - f_t(\mathbf{u})\right) &\le \sum_{t=1}^{T} \left(\frac{1}{2\gamma'} \left\|\boldsymbol{\nabla}_t\right\|_{A_t^{-1}}^2 + \frac{\gamma'}{2} \left\|\mathbf{y}_t - \mathbf{u}\right\|_{A_{t-1}}^2 - \frac{\gamma'}{2} \left\|\mathbf{y}_{t+1} - \mathbf{u}\right\|_{A_t}^2 \right) \\ &= \left(\frac{1}{2\gamma'} \sum_{t=1}^{T} \left\|\boldsymbol{\nabla}_t\right\|_{A_t^{-1}}^2 \right) + \frac{\gamma'}{2} \left\|\mathbf{y}_1 - \mathbf{u}\right\|_{A_0}^2 - \frac{\gamma'}{2} \left\|\mathbf{y}_{T+1} - \mathbf{u}\right\|_{A_T}^2 \\ &\overset{(17)}{\le} \frac{d}{2\gamma'} \log \left(1 + \frac{c_f^2 c_g^2 G^2}{d\epsilon} T \right) + \frac{\gamma' \epsilon D^2}{8}.\end{aligned}$$

The last inequality uses Lemma 5 and Condition 1, i.e., $\left\|\boldsymbol{\nabla}_t\right\|_2 = \left\|c_f \nabla g_t(\mathbf{y}_t)\right\|_2 \le c_f c_g G$.

Finally, the desired runtime of Eq. (10b) follows from the following two parts:

- **Runtime aside from FastProj.** In each round, the domain conversion of Lemma 3 takes $O(\mathsf{EP}_{\mathcal{X}} + d)$ time, updating and inverting A_t as Eq. (20) takes $O(d^2)$ time, and other operations, such as computing the surrogate gradient as Eq. (8b), take only $O(d)$ time. The overall runtime aside from FastProj is $O((\mathsf{EP}_{\mathcal{X}} + d^2)T)$.
- **Runtime of FastProj.** By Lemma 7, the number of bisections $n_t = O(\log t) = O(\log T)$ and each bisection takes $O(d^\omega)$ time with choice 1. Since LightONS.Core is a subroutine in LightONS, the number of calls to FastProj is at most $O((k-1)^{-1} d^{0.5} \sqrt{T/\epsilon})$ by Lemma 2. Specifically, since $A_t = \epsilon I + \sum_{i=1}^{t} \boldsymbol{\nabla}_i \boldsymbol{\nabla}_i^\mathsf{T}$ in Algorithm 3, we have

$$\Phi_T \triangleq \sum_{t=1}^{T} \left\| \frac{1}{\gamma'} A_t^{-1} \boldsymbol{\nabla}_t \right\|_2^2 \le \frac{1}{\gamma'^2} \frac{d}{\epsilon}, \quad N < \frac{2\sqrt{T\Phi_T}}{(k-1)D} \le \frac{2}{(k-1)D\gamma'} \sqrt{\frac{d}{\epsilon} T}.$$

  The first inequality uses Lemma 6, which controls Φ_T as a constant unrelated to the gradients; The second inequality follows from the proof of Lemma 2, which remains valid as long as the domain to project onto and the extended domain are separated by a margin of at least $(k-1)D/2$. The overall runtime of FastProj is $O((k-1)^{-1} d^{0.5} \sqrt{T/\epsilon} \cdot d^\omega \log T)$.

■

Appendix D. Proofs for Section 4

In this section, we provide the proofs and details for Section 4.

D.1. Proof of Theorem 3

Before proving Theorem 3, we first give a more detailed presentation of the convergence rates induced by LightONS. Throughout this subsection, we use the regime $\varepsilon = O(1/d)$ stated in Section 4.2. With $T = \Theta\left(\frac{d}{\varepsilon} \log \frac{d}{\varepsilon} \log \frac{1}{\delta}\right)$, we have

$$F(\bar{\mathbf{x}}_T) - \min_{\mathbf{x} \in \mathcal{X}} F(\mathbf{x}) \le \frac{1}{T} \left(\textsc{Reg}_T + 4 \sqrt{\frac{\textsc{Reg}_T}{2\gamma_0} \log \frac{4 \log T}{\delta}} + \frac{8}{\gamma_0} \log \frac{4 \log T}{\delta} \right) = O(\varepsilon), \tag{27a}$$

$$\textsc{Runtime} \le O\left(\frac{d^3}{\varepsilon} \log \frac{d}{\varepsilon} \log \frac{1}{\delta} + \frac{d^{3.5}}{\sqrt{\varepsilon}} \log \frac{d}{\varepsilon} \log \frac{1}{\delta} \right) = \widetilde{O}\left(\frac{d^3}{\varepsilon} \right), \tag{27b}$$

where $\textsc{Reg}_T$ follows from Eq. (11a) in Corollary 1. With $\frac{1}{\delta} = T' = \Theta\left(\frac{d}{\varepsilon}\log\frac{d}{\varepsilon}\right)$, we have

$$\mathbb{E}\left[F(\bar{\mathbf{x}}_{T'}) - \min_{\mathbf{x}\in\mathcal{X}} F(\mathbf{x})\right] \le O(\varepsilon), \tag{28a}$$

$$\textsc{Runtime} \le O\left(\frac{d^3}{\varepsilon}\log\frac{d}{\varepsilon} + \frac{d^{3.5}}{\sqrt{\varepsilon}}\log\frac{d}{\varepsilon}\right) = \widetilde{O}\left(\frac{d^3}{\varepsilon}\right). \tag{28b}$$

We remark that shifting the online loss function from $f_t(\mathbf{u}) = f(\mathbf{u};\xi_t)$ to $h_t(\mathbf{u}) = f(\frac{\mathbf{u}+\mathbf{x}_t}{2};\xi_t)$ improves the high-probability excess risk bound of Eq. (27a) to $\frac{2}{T}\left(\textsc{Reg}_T + \frac{2}{\gamma_0}\log\frac{1}{\delta}\right)$. This removes the $\log\log T$ term, though the asymptotic rate in Theorem 3 remains unchanged. We refer readers to Theorem 1 of van der Hoeven et al. [2023] for details.

Proof First, we prove Eq. (27a). Without loss of generality, we consider $T \ge 3$. Then the high-probability excess risk bound of Eq. (27a) directly follows from Corollary 2 of Mehta [2017] by substituting LightONS's regret in Corollary 1.

Then, we verify that the choice of $T = \Theta\left(\frac{d}{\varepsilon}\log\frac{d}{\varepsilon}\log\frac{1}{\delta}\right)$ yields a high-probability excess risk of $O(\varepsilon)$. Let $\Gamma_\delta = \frac{4\log T}{\delta} = O\left(\frac{1}{\delta}\log\frac{d}{\varepsilon} + \frac{1}{\delta}\log\log\frac{1}{\delta}\right)$. We have

$$F(\bar{\mathbf{x}}_T) - \min_{\mathbf{x}\in\mathcal{X}} F(\mathbf{x}) \le O\left(\frac{\textsc{Reg}_T + \sqrt{\textsc{Reg}_T\cdot\log\Gamma_\delta} + \log\Gamma_\delta}{T}\right) \le O\left(\frac{\textsc{Reg}_T + \log\Gamma_\delta}{T}\right).$$

The first inequality is essentially Corollary 2 of Mehta [2017], and the second inequality follows from the fact that $\sqrt{ab} \le \frac{a+b}{2} = O(a+b)$ for any positive terms a and b. Since $\textsc{Reg}_T = O(d\log T) = O\left(d\log\frac{d}{\varepsilon} + d\log\log\frac{1}{\delta}\right)$, it suffices to show that

$$O\left(\frac{\textsc{Reg}_T}{T}\right) = O\left(\frac{d\left(\log\frac{d}{\varepsilon} + \log\log\frac{1}{\delta}\right)}{\frac{d}{\varepsilon}\log\frac{d}{\varepsilon}\log\frac{1}{\delta}}\right) = O(\varepsilon), \quad \text{and}$$

$$O\left(\frac{\log\Gamma_\delta}{T}\right) = O\left(\frac{\log\frac{1}{\delta} + \log\left(\log\frac{d}{\varepsilon} + \log\log\frac{1}{\delta}\right)}{\frac{d}{\varepsilon}\log\frac{d}{\varepsilon}\log\frac{1}{\delta}}\right) = O\left(\frac{\varepsilon}{d}\right) = O(\varepsilon).$$

Next, we prove Eq. (28a) from Eq. (27a). By the definition of expectation, we have

$$\begin{aligned}\mathbb{E}\left[F(\bar{\mathbf{x}}_{T'}) - \min_{\mathbf{x}\in\mathcal{X}} F(\mathbf{x})\right] &\le \left(1 - \frac{1}{T'}\right)\cdot O\left(\frac{\textsc{Reg}_{T'} + \sqrt{\textsc{Reg}_{T'}\cdot\log\Gamma_{1/T'}} + \log\Gamma_{1/T'}}{T'}\right) \\ &\quad + \frac{1}{T'}\left(\max_{\mathbf{y}\in\mathcal{X}} F(\mathbf{y}) - \min_{\mathbf{x}\in\mathcal{X}} F(\mathbf{x})\right) \\ &\le O\left(\frac{\textsc{Reg}_{T'} + \log\Gamma_{1/T'}}{T'}\right) + \frac{DG}{T'} \le O\left(\frac{\textsc{Reg}_{T'} + \log\Gamma_{1/T'}}{T'}\right).\end{aligned}$$

The above inequalities use the Lipschitzness of F and the boundedness of $\mathcal{X}$. Similarly, we verify that the choice of $T' = \Theta\left(\frac{d}{\varepsilon}\log\frac{d}{\varepsilon}\right)$ yields an in-expectation excess risk of $O(\varepsilon)$ with $\frac{1}{\delta} = T' = \Theta\left(\frac{d}{\varepsilon}\log\frac{d}{\varepsilon}\right)$. It suffices to note that $\Gamma_{1/T'} = 4T'\log T' = O\left(\frac{d}{\varepsilon}\log^2\frac{d}{\varepsilon}\right)$ and that

$$O\left(\frac{\log\Gamma_{1/T'}}{T'}\right) = O\left(\frac{\log\frac{d}{\varepsilon}}{\frac{d}{\varepsilon}\log\frac{d}{\varepsilon}}\right) = O\left(\frac{\varepsilon}{d}\right) = O(\varepsilon).$$

Finally, the total runtime in Eqs. (27b) and (28b) follows from Theorem 2. ■

D.2. Proofs and Details for ERM-based SXO Methods

In this subsection, we construct an SXO instance where applying cutting-plane methods incurs $\widetilde{O}(d^3/\varepsilon)$ runtime, and we give an intuitive explanation of the runtime barrier $\widetilde{O}(d^3/\varepsilon)$.

Proposition 3 *Under Assumption 4 and that $\mathcal{X} = \mathcal{B}(1)$, let the stochastic functions take the form $f(\mathbf{x};\xi) = \phi\left(\mathbf{w}(\xi)^\mathsf{T}\mathbf{x}\right)$, where $\phi : \mathbb{R} \to \mathbb{R}$ is a black-box function, and $\mathbf{w} : \Xi \to \mathbb{R}^d$ is a fixed mapping. By [Koren and Levy, 2015; Mehta, 2017], obtaining an ε-optimal solution for SXO reduces to obtaining an $O(\varepsilon)$-optimal minimizer of the offline α-exp-concave objective*

$$\widehat{F}(\mathbf{x}) = \frac{1}{T}\sum_{t=1}^{T} \phi\left(\mathbf{w}(\xi_t)^\mathsf{T}\mathbf{x}\right),$$

where $T = \widetilde{O}(d/\varepsilon)$ is the necessary sample size. The best-known cutting-plane methods of Lee et al. [2015]; Jiang et al. [2020] solve this offline problem to $O(\varepsilon)$-accuracy in $\widetilde{O}(d^3/\varepsilon)$ time.

Proof of Proposition 3 Cutting-plane methods query the gradient of $\widehat{F}$ for $\widetilde{O}(d)$ times, and each gradient query costs $O(dT)$ time due to the finite-sum structure of $\widehat{F}$ and the black-box nature of ϕ. Therefore, the total runtime is $\widetilde{O}(d^2T + d^3) = \widetilde{O}(d^3/\varepsilon)$. ■

An intuitive explanation of the runtime $\widetilde{O}(d^3/\varepsilon)$. With the online-to-batch conversion, OGD's regret $O(\sqrt{T})$ for OCO translates to a sample complexity of $O(1/\varepsilon^2)$ and a runtime of $\widetilde{O}(d/\varepsilon^2)$ for SXO. When the target excess risk ε is relatively large, i.e., $\varepsilon = \widetilde{\Omega}(1/d^2)$, the OGD-based SXO method may circumvent the $\widetilde{O}(d^3/\varepsilon)$ runtime barrier. Thus the computational challenge of SXO mainly manifests when pursuing the (near) optimal sample complexity of $\widetilde{O}(d/\varepsilon)$.

Lemma 1 indicates that an exp-concave function can be regarded as a strongly convex function in the direction of its gradient. Comparing the sample complexity lower bound $\Omega(d/\varepsilon)$ for SXO [Mahdavi et al., 2015] to the $\Omega(1/\varepsilon)$ rate for stochastic strongly convex optimization [Hazan and Kale, 2011], the exp-concave offline objective F can be viewed as the sum of d strongly convex objectives on d directions, one along each dimension corresponding to the eigenvectors of the Hessian-related matrix A_t in ONS.

Our intuition is as follows: the difficulty of pursuing the (near) optimal sample complexity arises from exploiting the directional strong convexity, which necessitates estimating the covariance structure of the stochastic gradients. Cai et al. [2010] establish a minimax sample complexity for covariance matrix estimation under squared Frobenius norm, which coincides with the sample complexity lower bound for SXO. Specifically, for some family of covariance matrices $\mathcal{S}$, with T samples i.i.d. drawn from a distribution with covariance $\Sigma \in \mathcal{S}$, the estimator $\widehat{\Sigma}$ satisfies

$$\inf_{\widehat{\Sigma}} \sup_{\Sigma\in\mathcal{S}} \mathbb{E}\left[\|\widehat{\Sigma} - \Sigma\|_{\mathrm{F}}^2\right] = O\left(\frac{d}{T}\right).$$

We refer readers to Theorem 1 of Cai et al. [2010] for a formal statement. Since computing the sample covariance matrix requires $O(d^2T)$ time, estimating the covariance structure of stochastic gradients plausibly incurs at least $O(d^2T)$ runtime, leading to the $\widetilde{O}(d^3/\varepsilon)$ runtime barrier.

Appendix E. Proofs for Section 5

In this section, we provide algorithms and proofs for LightONS's applications listed in Section 5.

E.1. Proofs and Details for Gradient-Norm Adaptivity

In this subsection, we show how LightONS recovers ONS's gradient-norm adaptive regret bound in Eq. (12), and thereby leads to small-loss adaptivity for OXO with smoothness, as well as comparator-norm adaptivity for unbounded OCO. We state the theoretical guarantees based on LightONS and omit those of the original ONS-based methods as they are identical except for the runtime.

We say f is H-smooth on $\mathcal{X}$ if for any $(\mathbf{x}, \mathbf{y}) \in \mathcal{X}^2$, $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\|_2 \leq H \|\mathbf{x} - \mathbf{y}\|_2$.

Theorem 5 (LightONS's improvement for OXO with smoothness) *Under Assumptions 1–3, and that f_t is H-smooth for any $t \in [T]$, LightONS (Algorithm 3) satisfies that,*

$$\begin{aligned}
\textsc{Reg}_T &\leq \frac{d}{2\gamma_0} \log \left(\frac{8H}{d\epsilon} L_T + \frac{4H}{\gamma_0 \epsilon} \log \frac{4H}{\mathrm{e}\gamma_0 \epsilon} + \frac{\gamma_0 D^2 H}{d} + 2 \right) + \frac{\gamma_0 \epsilon D^2}{8}, \\
\textsc{Runtime} &\leq O\left(\left(\mathsf{EP}_{\mathcal{X}} + d^2 \right) T + d^{\omega} \sqrt{T} \log T \right),
\end{aligned}$$

where L_T is defined in Eq. (13), and γ_0 is defined in Corollary 1.

Proof of Theorem 5 Based on the proof of Theorem 1 of Orabona et al. [2012], it suffices to bound the gradient norms of the surrogate loss by those of the original loss. We note that Appendix B.4.2 of Yang et al. [2024] also discusses the small-loss bounds of ONS with domain conversion, although with a different domain conversion. Since $\|\nabla g_t(\mathbf{y}_t)\|_2 \leq \|\nabla f_t(\mathbf{x}_t)\|_2$ by Lemma 3, we have

$$G_{T,g} \triangleq \sum_{t=1}^{T} \|\nabla g_t(\mathbf{y}_t)\|_2^2 \leq \sum_{t=1}^{T} \|\nabla f_t(\mathbf{x}_t)\|_2^2 \triangleq G_{T,f}.$$

Theorem 2 implies the runtime with $\epsilon = d$ and the regret of

$$\textsc{Reg}_T(\mathbf{u}) \leq \frac{d}{2\gamma_0} \log \left(1 + \frac{G_{T,g}}{d\epsilon} \right) + \frac{\gamma_0 \epsilon D^2}{8} \leq \frac{d}{2\gamma_0} \log \left(1 + \frac{G_{T,f}}{d\epsilon} \right) + \frac{\gamma_0 \epsilon D^2}{8}.$$

The preceding inequality uses Jensen's inequality and $\operatorname{tr}(A_T) = d\epsilon + G_{T,g}$, i.e.,

$$\log \det(A_T) - \log \det(A_0) \leq d \log \frac{\operatorname{tr}(A_T)}{d} - \log \det(A_0) = d \log \left(1 + \frac{G_{T,g}}{d\epsilon} \right). \tag{29}$$

Then Corollary 5 of Orabona et al. [2012] converts the gradient-norm adaptive bound to the small-loss bound with respect to L_T and completes the proof. ■

Theorem 6 (LightONS's improvement for unbounded OCO) *If the loss function $f_t : \mathbb{R}^d \to \mathbb{R}$ is convex and 1-Lipschitz for any $t \in [T]$, then there exists an algorithm satisfying that, for any $\mathbf{u} \in \mathbb{R}^d$,*

$$\begin{aligned}
\textsc{Reg}_T(\mathbf{u}) &\leq \widetilde{O}\left(\sqrt{d \sum_{t=1}^{T} (\nabla f_t(\mathbf{x}_t)^{\mathsf{T}} \mathbf{u})^2} \right) \leq \widetilde{O}\left(\|\mathbf{u}\|_2 \sqrt{d G_T} \right), \\
\textsc{Runtime} &\leq O\left(d^2 T + d^{\omega} \sqrt{T} \log T \right).
\end{aligned}$$

Proof of Theorem 6 Theorem 8 of Cutkosky and Orabona [2018] hinges on their Algorithm 7, Lemmas 16 and 17, apart from the coin-betting framework. To prove Theorem 6, we show how LightONS adapts their analysis with minimal changes.

- **Modifications to their Algorithm 7.** Since the decisions of their ONS are intermediate decisions to maximize the "wealth" in the coin-betting framework instead of true decisions to minimize regret, we can ignore the improper-to-proper conversion and replace their ONS with LightONS.Core. Specifically, their ONS runs on $\mathcal{X} = \mathcal{B}(1/2)$ while LightONS.Core runs on $\mathcal{Y} = \mathcal{B}(3/4)$ with the deferral coefficient $k = 3/2$.

- **Modifications to their Lemma 16.** Their ONS's domain $\mathcal{X} = \mathcal{B}(1/2)$ implies a curvature parameter $\gamma = \frac{2-\log 3}{2}$ while LightONS.Core's domain $\mathcal{Y} = \mathcal{B}(3/4)$ implies $\gamma = \frac{6-\log 7}{18}$.

- **Modifications to their Lemma 17.** Relaxing the radius from $1/2$ to $3/4$ enlarges constants in the regret. Nonetheless, the numerical constants in their Lemma 17 are loose enough to accommodate our changes, greatly simplifying our analysis. Following their proof, let $\mathbf{z}_t$ denote the gradient that LightONS.Core receives at time t, Theorem 2 and Eq. (29) imply

$$\textsc{Reg}_T \le \frac{d}{2\gamma} \log\left(1 + \frac{\sum_{t=1}^T \|\mathbf{z}_t\|_2^2}{d\epsilon}\right) + \frac{\gamma\epsilon D^2}{8}.$$

  Plugging $D = 1$, $\gamma = \frac{6-\log 7}{18} \in (0.225, 0.226)$, $\epsilon = d$, and $\|\mathbf{z}_t\|_2^2 \le 16\,\|\nabla f_t(\mathbf{x}_t)\|_2^2$ yields

$$\textsc{Reg}_T \le d\left(\frac{5}{2}\log\left(1 + \frac{16}{d^2}\sum_{t=1}^T \|\nabla f_t(\mathbf{x}_t)\|_2^2\right) + \frac{1}{35}\right).$$

  The preceding regret bound fully recovers their Lemma 17 when $d \ge 2$.

The runtime follows from Theorem 2 with $\epsilon = d$. ■

Comparison with OQNS. We remark that OQNS can hardly achieve full gradient-norm adaptivity, as the log-barrier regularization introduces an unavoidable $O(\log T)$ bias term independent of G_T. From the perspective of OMD, the regret can be decomposed into a stability term and a bias term. Adopted from Hazan [2016]; Orabona [2019], the regret decomposition of a typical OMD algorithm is as follows:

$$\textsc{Reg}_T(\mathbf{u}) \le \underbrace{\frac{\eta}{2}\sum_{t=1}^T \|\nabla f_t(\mathbf{x}_t)\|_*^2}_{\text{stability}} + \underbrace{\frac{\|\mathbf{u}-\mathbf{x}_1\|^2}{2\eta}}_{\text{bias}}. \tag{30}$$

While ONS and LightONS, as instances of OMD, can flexibly balance stability and bias, [3] OQNS sacrifices such flexibility for computational efficiency. OQNS employs a highly stable log-barrier, which suppresses the stability term but inflates the bias term in a manner resistant to small gradient norms. Specifically, the log-barrier contributes a term $-\log(D^2 - \|\mathbf{u}\|_2^2)$ in the regret for any

3. For ONS and LightONS, the learning rate η is replaced with the time-varying Hessian-related matrix's inverse A_t^{-1}.

comparator $\mathbf{u} \in \mathcal{B}(D)$. As $\|\mathbf{u}\|_2$ approaches D, this term diverges to infinity. This issue is mitigated with the fixed-share trick [Mhammedi and Gatmiry, 2023], an illustration of which is as follows:

$$\sum_{t=1}^{T}(f_t(\mathbf{x}_t)-f_t(\mathbf{u})) \le \underbrace{\sum_{t=1}^{T}(f_t(\mathbf{x}_t)-f_t(\mathbf{v}))}_{\text{fixed-share regret}} + \underbrace{\sum_{t=1}^{T}(f_t(\mathbf{v})-f_t(\mathbf{u}))}_{\text{fixed-share margin}}, \quad \text{where } \mathbf{v} = \left(1-\frac{1}{T}\right)\mathbf{u}.$$

The fixed-share margin term $f_t(\mathbf{v}) - f_t(\mathbf{u}) \le O(GD/T)$ due to boundedness and Lipschitzness, summing to $O(1)$ over T rounds. Since $1-\|\mathbf{v}\|_2^2 \ge \Omega(D^2/T)$, the fixed-share regret term contributes $-\log(D^2 - \|\mathbf{v}\|_2^2) = O(\log T)$ to the regret, which prevents full gradient-norm adaptivity.

E.2. Proofs and Details for Logistic Bandits

In this subsection, we focus on logistic bandits as a representative instance of generalized linear bandits (GLB) to concretely illustrate the applicability of LightONS.

Problem setting. Logistic bandits can be interpreted as interactions between a learner and an adversary, which unfolds as follows: At each round $t \in [T]$, the learner selects an arm $\mathbf{x}_t$ from an arm set $\mathcal{X}_t \subseteq \mathcal{B}(1)^K$ with K arms, and suffers a stochastic loss $y_t \in \{0,1\}$, where $\mathbb{P}(y_t = 1 \mid \mathbf{x}_t) = \sigma(\mathbf{x}_t^\mathsf{T}\mathbf{w}^\star)$. The true parameter $\mathbf{w}^\star \in \mathcal{W} = \mathcal{B}(D/2)$ is unknown, and $\sigma(z) = 1/(1+\exp(-z))$ denotes the sigmoid function. The performance of the learner is measured by its pseudo-regret, quantifying the cumulative expected loss against the optimal arms in hindsight, which is defined as

$$\textsc{Reg}_T = \sum_{t=1}^{T}\left(\sigma(\mathbf{x}_t^\mathsf{T}\mathbf{w}^\star) - \sigma(\mathbf{x}_t^{\star\mathsf{T}}\mathbf{w}^\star)\right),$$

where $\mathbf{x}_t^\star = \arg\max_{\mathbf{x}\in\mathcal{X}_t}\mathbf{x}^\mathsf{T}\mathbf{w}^\star$.

A challenge in GLB lies in its dependence on $\kappa = \max_{\mathbf{x}\in\mathcal{X},\mathbf{w}\in\mathcal{W}} 1/\sigma'(\mathbf{x}^\mathsf{T}\mathbf{w})$, a problem-dependent constant that may grow exponentially with D.

Jointly efficient GLB algorithm. Zhang et al. [2025] propose the first one-pass (constant memory and constant per-round time) GLB algorithm achieving κ-free-leading-term pseudo-regret. Their pseudo-regret is $O(d\sqrt{T}\log T + \kappa(d\log T)^2)$, which remains sublinear in T for $\kappa = o(T)$.

Their key idea is to estimate the parameter $\mathbf{w}^\star$ with a "look-ahead" variant of ONS, whose analytical properties can remove the dependence on κ in the pseudo-regret's leading terms while retaining the constant-memory and constant-time efficiency of ONS. An illustration of their "look-ahead" ONS is provided as follows:

$$\begin{aligned}
\widetilde{H}_t &= \lambda I + \left(\sum_{i=1}^{t-1}\nabla^2\ell_i(\mathbf{w}_{i+1})\right) + \eta\nabla^2\ell_t(\mathbf{w}_t),\\
\widehat{\mathbf{w}}_{t+1} &= \mathbf{w}_t - \frac{1}{\eta}\widetilde{H}_t^{-1}\nabla\ell_t(\mathbf{w}_t),\\
\mathbf{w}_{t+1} &= \Pi^{\widetilde{H}_t}_{\mathcal{B}(D/2)}[\widehat{\mathbf{w}}_{t+1}],
\end{aligned} \tag{31}$$

where $\ell_t(\mathbf{w}) = -y_t\log\sigma(\mathbf{x}_t^\mathsf{T}\mathbf{w}) - (1-y_t)\log(1-\sigma(\mathbf{x}_t^\mathsf{T}\mathbf{w}))$ denotes the logistic loss for parameter estimation at round t.

Algorithm 5 "look-ahead" LightONS for [Zhang et al., 2025]

Input: domain $\mathcal{W} = \mathcal{B}(D/2)$, regularization coefficient λ, inverse step size η.

1: Initialize $H_0 = \lambda I$, $\mathbf{w} = \mathbf{0}$.
2: **for** $t = 1, \dots, T$ **do**
3: Update the lower confidence bound function as in Algorithm 1 of Zhang et al. [2025].
4: Select the arm $\mathbf{x}_t$ as in Algorithm 1 of Zhang et al. [2025] and observe the loss y_t.
5: $\widetilde{H}_t = H_{t-1} + \eta \nabla^2 \ell_t(\mathbf{w}_t)$.
6: $\widehat{\mathbf{w}}_{t+1} = \mathbf{w}_t - \frac{1}{\eta} \widetilde{H}_t^{-1} \nabla \ell_t(\mathbf{w}_t)$.
7: $\mathbf{w}_{t+1} = \begin{cases} \widehat{\mathbf{w}}_{t+1} & \text{if } \|\widehat{\mathbf{w}}_{t+1}\|_2 \le k^\star D/2 \\ \Pi^{\widetilde{H}_t}_{\mathcal{B}(D/2)}[\widehat{\mathbf{w}}_{t+1}] & \text{otherwise} \end{cases}$, where $k^\star = 1 + \frac{2}{\log \kappa} = \Theta(1 + 1/D)$.
8: $H_t = H_{t-1} + \nabla^2 \ell_t(\mathbf{w}_{t+1})$.
9: **end for**

Because of ONS, their algorithm incurs the worst-case $\widetilde{O}((d^2K + d^\omega)T)$ runtime bottleneck, where K is the number of arms. Below we show that plugging LightONS can improve the runtime while preserving the κ-free-leading-term pseudo-regret guarantee.

LightONS-based counterpart. We propose an analogous "look-ahead" variant of LightONS in Algorithm 5, which replaces the ONS-based subroutine in Zhang et al. [2025]. Line 7 of Algorithm 5 introduces the deferred-projection mechanism of LightONS.Core to replace the Mahalanobis projection in Eq. (31).

Algorithm 5 improves the runtime to $\widetilde{O}(d^2KT + d^\omega \cdot \min\{\sqrt{\kappa dT}\log\kappa, T\})$, while preserving the κ-free-leading-term pseudo-regret guarantee. When $\kappa = o(T)$, as required for sublinear pseudo-regret, the improved algorithm is asymptotically faster than the original algorithm. We state the theoretical guarantee of Algorithm 5 in the theorem below.

Theorem 7 (LightONS's improvement for logistic bandits) *For binary logistic bandits, under the conditions $\bigcup_{t=1}^T \mathcal{X}_t \subseteq \mathcal{B}(1)$, and $\|\mathbf{w}^\star\| \le D/2$ where D is known a priori, there exists an algorithm achieving a pseudo-regret of $O(d\sqrt{T}\log T + \kappa(d\log T)^2)$, and a total runtime of $\widetilde{O}(d^2KT + d^\omega \cdot \min\{\sqrt{\kappa dT}\log\kappa, T\})$.*

We remark that Algorithm 5 does not suffer from improper learning concerns, because the algorithm's decision is the arm $\mathbf{x}_t$, rather than the estimated parameter $\mathbf{w}_t$. Consequently, Algorithm 5 is free from the improper-to-proper conversion and resembles LightONS.Core.

Before proving Theorem 7, we first present two critical lemmas that facilitate migrating the original ONS-based subroutine to LightONS-based Algorithm 5. The following lemma resembles our Lemma 11 and Lemma 1 of Zhang et al. [2025], showing that Algorithm 5 implies an OMD-like regret decomposition form similar to that of the original ONS-based subroutine.

Lemma 13 *Algorithm 5 satisfies that, for any $\mathbf{u} \in \mathcal{B}(D/2)$,*

$$\|\mathbf{w}_{t+1} - \mathbf{u}\|^2_{H_{t-1}} \le 2\eta \nabla \widetilde{\ell}_t(\mathbf{w}_{t+1})^\mathsf{T}(\mathbf{u} - \mathbf{w}_{t+1}) + \|\mathbf{w}_t - \mathbf{u}\|^2_{H_{t-1}} - \|\mathbf{w}_t - \mathbf{w}_{t+1}\|^2_{H_{t-1}},$$

where $\widetilde{\ell}_t$ is the second-order Taylor expansion of ℓ_t at $\mathbf{w}_t$, i.e.,

$$\widetilde{\ell}_t(\mathbf{w}) \triangleq \ell_t(\mathbf{w}_t) + \nabla \ell_t(\mathbf{w}_t)^\mathsf{T}(\mathbf{w} - \mathbf{w}_t) + \frac{1}{2}\|\mathbf{w} - \mathbf{w}_t\|^2_{\nabla^2 \ell_t(\mathbf{w}_t)}.$$

Proof of Lemma 13 We note that the descent-and-projection update in Algorithm 5 is equivalent to the OMD update, as Appendix D of Zhang et al. [2025] has shown. Specifically,

$$\begin{aligned} \mathbf{w}_{t+1} &= \Pi^{\widetilde{H}_t}_{\mathcal{B}(D/2)}[\widehat{\mathbf{w}}_{t+1}] = \Pi^{\widetilde{H}_t}_{\mathcal{B}(D/2)}\left[\mathbf{w}_t - \frac{1}{\eta}\widetilde{H}_t^{-1}\nabla\ell_t(\mathbf{w}_t)\right] \\ \iff \mathbf{w}_{t+1} &= \operatorname*{arg\,min}_{\mathbf{w}\in\mathcal{B}(D/2)} \widetilde{\ell}_t(\mathbf{w}) + \frac{1}{2\eta}\left\|\mathbf{w}-\mathbf{w}_t\right\|^2_{H_{t-1}}, \end{aligned} \tag{32}$$

and

$$\begin{aligned} \mathbf{w}_{t+1} &= \widehat{\mathbf{w}}_{t+1} = \mathbf{w}_t - \frac{1}{\eta}\widetilde{H}_t^{-1}\nabla\ell_t(\mathbf{w}_t) \\ \iff \mathbf{w}_{t+1} &= \operatorname*{arg\,min}_{\mathbf{w}\in\mathbb{R}^d} \widetilde{\ell}_t(\mathbf{w}) + \frac{1}{2\eta}\left\|\mathbf{w}-\mathbf{w}_t\right\|^2_{H_{t-1}}. \end{aligned} \tag{33}$$

To recover Lemma 1 of Zhang et al. [2025], we examine whether the following inequality holds

$$\nabla\widetilde{\ell}_t(\mathbf{w}_{t+1})^\mathsf{T}(\mathbf{w}_{t+1}-\mathbf{u}) \le \frac{1}{2\eta}\left(\left\|\mathbf{w}_t-\mathbf{u}\right\|^2_{H_{t-1}} - \left\|\mathbf{w}_{t+1}-\mathbf{u}\right\|^2_{H_{t-1}} - \left\|\mathbf{w}_{t+1}-\mathbf{w}_t\right\|^2_{H_{t-1}}\right). \tag{34}$$

When $\|\widehat{\mathbf{w}}_{t+1}\|_2 \le k^\star D/2$ and the Mahalanobis projection is not triggered, by Eq. (33) we have

$$\nabla_{\mathbf{w}=\mathbf{w}_{t+1}}\left(\widetilde{\ell}_t(\mathbf{w}) + \frac{1}{2\eta}\left\|\mathbf{w}-\mathbf{w}_t\right\|^2_{H_{t-1}}\right)^\mathsf{T}(\mathbf{w}_{t+1}-\mathbf{u}) = \mathbf{0}^\mathsf{T}(\mathbf{w}_{t+1}-\mathbf{u}) = 0.$$

Rearranging terms, we have

$$\begin{aligned} \nabla\widetilde{\ell}_t(\mathbf{w}_{t+1})^\mathsf{T}(\mathbf{w}_{t+1}-\mathbf{u}) &= -\frac{1}{\eta}(\mathbf{w}_{t+1}-\mathbf{w}_t)^\mathsf{T}H_{t-1}(\mathbf{w}_{t+1}-\mathbf{u}) \\ &= \frac{1}{2\eta}\left(\left\|\mathbf{w}_t-\mathbf{u}\right\|^2_{H_{t-1}} - \left\|\mathbf{w}_{t+1}-\mathbf{u}\right\|^2_{H_{t-1}} - \left\|\mathbf{w}_{t+1}-\mathbf{w}_t\right\|^2_{H_{t-1}}\right), \end{aligned}$$

which means that Eq. (34) holds with equality.

When $\|\widehat{\mathbf{w}}_{t+1}\|_2 > k^\star D/2$ and the Mahalanobis projection is triggered, by Eq. (32), we have

$$\nabla_{\mathbf{w}=\mathbf{w}_{t+1}}\left(\widetilde{\ell}_t(\mathbf{w}) + \frac{1}{2\eta}\left\|\mathbf{w}-\mathbf{w}_t\right\|^2_{H_{t-1}}\right)^\mathsf{T}(\mathbf{w}_{t+1}-\mathbf{u}) \le 0.$$

Rearranging terms, we have

$$\begin{aligned} \nabla\widetilde{\ell}_t(\mathbf{w}_{t+1})^\mathsf{T}(\mathbf{w}_{t+1}-\mathbf{u}) &\le -\frac{1}{\eta}(\mathbf{w}_{t+1}-\mathbf{w}_t)^\mathsf{T}H_{t-1}(\mathbf{w}_{t+1}-\mathbf{u}) \\ &= \frac{1}{2\eta}\left(\left\|\mathbf{w}_t-\mathbf{u}\right\|^2_{H_{t-1}} - \left\|\mathbf{w}_{t+1}-\mathbf{u}\right\|^2_{H_{t-1}} - \left\|\mathbf{w}_{t+1}-\mathbf{w}_t\right\|^2_{H_{t-1}}\right), \end{aligned}$$

which means that Eq. (34) holds.

Therefore, combining both cases, we have that Eq. (34) always holds. ■

The next lemma gives the total runtime of Algorithm 5.

Lemma 14 *Algorithm 5 has a runtime of* $\widetilde{O}(d^2T + d^\omega \cdot \min\{\sqrt{\kappa dT}\log\kappa, T\})$.

Proof of Lemma 14 We first show that the expansion of the domain influences the condition number κ. We recall that $\ell_t(\mathbf{w}) = -y_t \log \sigma(\mathbf{x}_t^\mathsf{T}\mathbf{w}) - (1-y_t)\log(1-\sigma(\mathbf{x}_t^\mathsf{T}\mathbf{w}))$ is the logistic loss function, with $\nabla \ell_t(\mathbf{w}) = \big(\sigma(\mathbf{x}_t^\mathsf{T}\mathbf{w}) - y_t\big)\,\mathbf{x}_t$ and $\nabla^2 \ell_t(\mathbf{w}) = \sigma'(\mathbf{x}_t^\mathsf{T}\mathbf{w})\mathbf{x}_t\mathbf{x}_t^\mathsf{T}$, where $\sigma(z) = 1/(1+\exp(-z))$ is the sigmoid function. By the definition,

$$\kappa = \max_{\mathbf{x}\in\mathcal{X},\mathbf{w}\in\mathcal{W}} \frac{1}{\sigma'(\mathbf{x}^\mathsf{T}\mathbf{w})} \le \max_{\|\mathbf{x}\|_2\le 1,\|\mathbf{w}\|_2\le D/2} (1+\exp(\mathbf{x}^\mathsf{T}\mathbf{w}))(1+\exp(-\mathbf{x}^\mathsf{T}\mathbf{w})) \le 2+2\exp(D/2).$$

Therefore, the condition number κ grows exponentially with the diameter of the domain. For technical convenience, we write $\kappa' = \kappa^k$ when the diameter of the domain is expanded from D to $D' = kD$ for some $k > 1$.

Compared with the runtime of LightONS Theorem 2, the only difference is that κ appears in the number of Mahalanobis projections. We note that H_t and $\widetilde{H}_t$ also admit rank-one updates. It suffices to show how the logistic loss function affects the analysis of Lemma 2. Specifically, we need to bound the quantity

$$\Phi'_T \triangleq \sum_{t=1}^{T} \left\| \frac{1}{\eta} \widetilde{H}_t^{-1} \nabla \ell_t(\mathbf{w}_t) \right\|_2^2.$$

By the update rule of Algorithm 5 and that $\eta > 1$ in [Zhang et al., 2025], we have

$$\widetilde{H}_t = \lambda I + \sum_{i=1}^{t-1} \sigma'(\mathbf{x}_i^\mathsf{T}\mathbf{w}_{i+1})\mathbf{x}_i\mathbf{x}_i^\mathsf{T} + \eta\sigma'(\mathbf{x}_t^\mathsf{T}\mathbf{w}_t)\mathbf{x}_t\mathbf{x}_t^\mathsf{T} \succ \lambda I + \sum_{i=1}^{t} \frac{1}{\kappa^k}\mathbf{x}_i\mathbf{x}_i^\mathsf{T}.$$

Since $\big|\sigma(\mathbf{x}_t^\mathsf{T}\mathbf{w}_t) - y_t\big| \le 1$, we have

$$\begin{aligned}\left\|\widetilde{H}_t^{-1}\nabla\ell_t(\mathbf{w}_t)\right\|_2^2 &= \left\|\widetilde{H}_t^{-1}(\sigma(\mathbf{x}_t^\mathsf{T}\mathbf{w}_t) - y_t)\mathbf{x}_t\right\|_2^2 \le \left\|\widetilde{H}_t^{-1}\mathbf{x}_t\right\|_2^2 \\ &< \left\|\left(\lambda I + \sum_{i=1}^{t}\frac{1}{\kappa^k}\mathbf{x}_i\mathbf{x}_i^\mathsf{T}\right)^{-1}\mathbf{x}_t\right\|_2^2 = \kappa^{2k}\left\|\left(\kappa^k\lambda I + \sum_{i=1}^{t}\mathbf{x}_i\mathbf{x}_i^\mathsf{T}\right)^{-1}\mathbf{x}_t\right\|_2^2.\end{aligned}$$

Summing up the preceding inequality with Lemma 6, we have

$$\Phi'_T < \frac{\kappa^{2k}}{\eta^2}\sum_{t=1}^{T}\left\|\left(\kappa^k\lambda I + \sum_{i=1}^{t}\mathbf{x}_i\mathbf{x}_i^\mathsf{T}\right)^{-1}\mathbf{x}_t\right\|_2^2 \le \frac{\kappa^k d}{\eta^2\lambda}.$$

Therefore, reusing the proof of Lemma 2, we obtain that the runtime of Algorithm 5 with deferral coefficient k is $\widetilde{O}\big(d^2T + d^\omega \cdot \min\{\frac{\sqrt{\kappa^k dT}}{k-1}, T\}\big)$. Let $\Gamma(k) = \frac{\kappa^{k/2}}{k-1}$, then minimizing $\Gamma(k)$ over $k > 1$ yields $k^\star = 1 + \frac{2}{\log\kappa}$, $\kappa^k = \mathrm{e}^2\kappa$, and $\Gamma(k^\star) = \frac{\mathrm{e}}{2}\sqrt{\kappa}\log\kappa$. ■

Based on the preceding lemmas, we are ready to prove Theorem 7.

Proof of Theorem 7 The upper-confidence-bound-based pseudo-regret analysis in [Zhang et al., 2025] primarily relies on their Theorem 1, which constitutes their Lemmas 4–6. To prove Theorem 7, we examine how replacing the ONS-based subroutine with LightONS-based Algorithm 5 affects these lemmas.

- **Modifications to their Lemma 4.** Their Lemma 4 is supported by their Lemma 1 and local relaxation of generalized linear models. Lemma 13 shows that their Lemma 1 still holds when using Algorithm 5 as a replacement. The local relaxation is independent of the specific update rules of $\mathbf{w}_t$ and remains valid.

- **Modifications to their Lemmas 5 and 6.** Their Lemmas 5 and 6 depend solely on the algebraic structure of the covariance matrix, i.e., $H_t = \lambda I + \sum_{i=1}^{t} \sigma'(\mathbf{x}_i^\mathsf{T}\mathbf{w}_{i+1})\mathbf{x}_i\mathbf{x}_i^\mathsf{T}$. Therefore, Algorithm 5 does not affect their Lemmas 5 and 6.

Therefore, the pseudo-regret analysis of Zhang et al. [2025] still holds when replacing their ONS-based subroutine with Algorithm 5, although with an expanded domain diameter $D' = k^\star D$ and a correspondingly larger condition number $\kappa' = \kappa^{k^\star} = \mathrm{e}^2\kappa = O(\kappa)$. Finally, the runtime of Algorithm 5 follows from Lemma 14. With the overhead $O(d^2KT)$ for selecting arms as in [Zhang et al., 2025], the total runtime is $\widetilde{O}(d^2KT + d^\omega \cdot \min\{\sqrt{\kappa dT}\log\kappa, T\})$. ■

We remark that extending Algorithm 5 from the binary logistic bandits to the generalized linear bandits directly follows from [Zhang et al., 2025].

Comparison with OQNS. OQNS can hardly be integrated into the method of [Zhang et al., 2025], as it is tailored to the OXO protocol and does not admit a regret decomposition analogous to Lemma 13, which is essential for eliminating κ from leading terms in pseudo-regret. Moreover, OQNS lacks the flexibility to accommodate customized local norms beyond those used in standard ONS. Specifically, the local norm $\widetilde{H}_t$ in [Zhang et al., 2025], i.e., Eq. (31), is not a simple accumulation of gradient outer products, unlike OQNS's local norm in Eq. (4). In contrast, LightONS retains the structural flexibility of ONS, enabling integration into GLB with minimal modifications.

E.3. Proofs and Details for Memory-Efficient OXO

In this subsection, we seek to reduce the working memory required to achieve the optimal $O(d\log T)$ regret in OXO, albeit conditioned on geometric properties of the problem instance.

First, we list the additional assumptions of Luo et al. [2016] as follows:

- **Additional assumption on domains.** For any $t \in [T]$, the domain at the t-th round is the intersection of two parallel half-spaces, i.e.,

$$\mathcal{X}_t = \{\mathbf{x} \,|\, |\mathbf{w}_t^\mathsf{T}\mathbf{x}| < D/2\}, \quad \text{where } \mathbf{w}_t \in \mathbb{R}^d \text{ is known and } \|\mathbf{w}_t\| = 1. \tag{35}$$

- **Additional assumption on loss functions.** For any $t \in [T]$, the curvature parameter γ, the loss function f_t, the trajectory $\{\mathbf{x}_t\}_{t=1}^T$ and the comparator $\mathbf{u} \in \bigcap_{t=1}^T \mathcal{X}_t$ satisfy

$$f_t(\mathbf{x}) - f_t(\mathbf{u}) \le \nabla f_t(\mathbf{x})^\mathsf{T}(\mathbf{x}-\mathbf{u}) - \frac{\gamma}{2}\left(\nabla f_t(\mathbf{x})^\mathsf{T}(\mathbf{x}-\mathbf{u})\right)^2. \tag{36}$$

Next, we formally state the theoretical guarantees of SON in the following proposition.

Algorithm 6 LightONS.Sketch

Input: preconditioner coefficient ϵ, dimension to reduce to d'.

Initialize $\gamma_0 = \frac{1}{2}\min\{\frac{1}{DG}, \alpha\}$, $S_0 = O_{2d' \times d}$, $R_0 = \frac{1}{\epsilon} I_{2d' \times 2d'}$, $\mathbf{x}_1 = \mathbf{y}_1 = \mathbf{0}$.
for $t = 1, \ldots, T$ **do**
  Observe $\nabla f_t(\mathbf{x}_t)$; and construct $\nabla g_t(\mathbf{y}_t)$ as in Lemma 3.
  Sketch $\nabla g_t(\mathbf{y}_t)$ into S_t and R_t with Algorithm 7. ▷ $\widetilde{A}_t \preceq \widetilde{A}_{t-1} + \nabla g_t(\mathbf{y}_t) \nabla g_t(\mathbf{y}_t)^\mathsf{T}$.
  $\widehat{\mathbf{y}}_{t+1} = \mathbf{y}_t - \frac{1}{\gamma_0} \widetilde{A}_t^{-1} \nabla g_t(\mathbf{y}_t)$. ▷ $\widetilde{A}_t^{-1} = \frac{1}{\epsilon}(I - S_t^\mathsf{T} R_t S_t)$.
  $\mathbf{y}_{t+1} = \begin{cases} \widehat{\mathbf{y}}_{t+1} & \text{if } \|\widehat{\mathbf{y}}_{t+1}\|_2 \le D \\ \Pi^{\widetilde{A}_t}_{\mathcal{B}(D/2)}[\widehat{\mathbf{y}}_{t+1}] & \text{otherwise} \end{cases}$. ▷ $\widetilde{A}_t = \epsilon I + S_t^\mathsf{T} S_t$.
  $\mathbf{x}_{t+1} = \Pi_{\mathcal{X}}[\mathbf{y}_{t+1}]$.
end for

Algorithm 7 Fast Frequent Directions of Ghashami et al. [2016]

Input: new gradient $\nabla g_t(\mathbf{y}_t)$, frequent directions S_{t-1}, low-dimension inverse R_{t-1}.
Output: updated frequent directions S_t, updated low-dimension inverse R_t.

$S_t = S_{t-1} + \mathbf{e}_{i_t} \nabla g_t(\mathbf{y}_t)^\mathsf{T}$, where i_t is the index of the first all-zero row of S_{t-1}.
if S_t still has all-zero rows **then**
  $R_t = \left(\epsilon I + S_t S_t^\mathsf{T}\right)^{-1}$.
else
  $U_t \Sigma_t V_t^\mathsf{T} = S_t$, truncated SVD with top d' singular values.
  $S_t = \begin{bmatrix} (\Sigma_t^2 - (\Sigma_t)^2_{d',d'} I)^{1/2} V_t^\mathsf{T} \\ O_{d' \times d} \end{bmatrix}$.
  $R_t = \operatorname{diag}\left(\frac{1}{\epsilon + (\Sigma_t)^2_{1,1} - (\Sigma_t)^2_{d',d'}}, \ldots, \frac{1}{\epsilon + (\Sigma_t)^2_{d',d'} - (\Sigma_t)^2_{d',d'}}, \frac{1}{\epsilon}, \ldots, \frac{1}{\epsilon} \right)$.
end if

Proposition 4 (Theorem 3 of Luo et al. [2016]) *Under Assumption 2 and additional assumptions described in Eqs. (35) and (36), SON (Algorithms 1 and 6 of Luo et al. [2016]) satisfies that,* [4]

$$\textsc{Reg}_T(\mathbf{u}) \le \frac{d'}{\gamma} \log\left(1 + \frac{G^2}{2d'\epsilon} T\right) + \frac{\gamma \epsilon D^2}{8} + \frac{\Delta_{1:T}}{2\gamma}, \tag{37a}$$

$$\textsc{Runtime} \le O\left(d' d T \log T\right), \tag{37b}$$

and the working memory is $O(d'd)$. The cumulative sketching error $\Delta_{1:T}$ can be bounded as

$$\Delta_{1:T} \le \min_{j \in [d']} \frac{2d'}{(d' - j + 1)\epsilon} \sum_{i=j}^{d} \lambda_i \left(\sum_{t=1}^{T} \nabla f_t(\mathbf{x}_t) \nabla f_t(\mathbf{x}_t)^\mathsf{T} \right). \tag{38}$$

Then, we present the LightONS.Sketch algorithm in Algorithm 6, a memory-efficient variant of LightONS. The key difference between LightONS.Sketch and LightONS is the storage strategy. Instead of storing full matrices $A_t \in \mathbb{R}^{d \times d}$ and $V_t \in \mathbb{R}^{d \times d}$, LightONS.Sketch maintains compact

4. The $\log T$ term in SON's runtime arises from the eigendecomposition underlying sketching. Although Theorem 3 of Luo et al. [2016] ignores it, we include it for comparison with LightONS and OQNS.

sketches $S_t \in \mathbb{R}^{2d' \times d}$ and $R_t \in \mathbb{R}^{2d' \times 2d'}$. Each row of S_t stores a principal gradient component, while R_t plays a role analogous to V_t in Eq. (20). The full matrix and its inverse are reconstructed as

$$\widetilde{A}_t = \epsilon I + S_t^\mathsf{T} S_t, \quad \widetilde{A}_t^{-1} = \frac{1}{\epsilon}\left(I - S_t^\mathsf{T} R_t S_t\right), \quad R_t = \left(\epsilon I + S_t S_t^\mathsf{T}\right)^{-1}.$$

This relationship follows from the Sherman-Morrison-Woodbury formula:

$$(A + BCD)^{-1} = A^{-1} - A^{-1}B(C^{-1} + DA^{-1}B)^{-1}DA^{-1}.$$

Following Luo et al. [2016], LightONS.Sketch uses Fast Frequent Directions of Ghashami et al. [2016] in Algorithm 7 to update S_t and R_t.

Before proving Theorem 4, we first introduce three lemmas to characterize the error introduced by sketching. We introduce the notation Δ_t to denote the sketching error at the t-th round.

$$\Delta_t \triangleq \begin{cases} \frac{2d'}{\epsilon}\sigma_{d'}(S_t)^2 & \text{if SVD is triggered at } t\text{-th round} \\ 0 & \text{otherwise} \end{cases},$$

where $\sigma_i(S_t)$ is the i-th greatest singular value of S_t. Specifically, Lemma 15 bounds the error for regret analysis with Lemma 5, while Lemma 16 bounds the error for projection-count analysis with Lemma 6. Lemma 17 bounds the total error with the spectrum of the Hessian-related matrix.

Lemma 15 *At the t-th round of Algorithm 6, in Algorithm 7, Δ_t satisfies:*

$$\|\nabla g_t(\mathbf{y}_t)\|^2_{\widetilde{A}_t^{-1}} \le \left\langle \widetilde{A}_t^{-1}, \widetilde{A}_t - \widetilde{A}_{t-1} \right\rangle_{\mathrm{F}} + \Delta_t.$$

Proof of Lemma 15 Let $\langle \cdot, \cdot \rangle_{\mathrm{F}}$ denote the inner product induced by the Frobenius matrix norm, let $(\cdot)_{:,i}$ denote the i-th column of the matrix, and let $\bar{U}_t \bar{\Sigma}_t \bar{V}_t^\mathsf{T} = S_t$ be the full SVD with all $2d'$ singular values. When SVD is not triggered, then $\widetilde{A}_t = \widetilde{A}_{t-1} + \nabla g_t(\mathbf{y}_t)\nabla g_t(\mathbf{y}_t)^\mathsf{T}$ and $\Delta_t = 0$ trivially holds. When SVD is triggered,

$$\begin{aligned} &\left\langle \widetilde{A}_t^{-1}, \widetilde{A}_{t-1} + \nabla g_t(\mathbf{y}_t)\nabla g_t(\mathbf{y}_t)^\mathsf{T} - \widetilde{A}_t \right\rangle_{\mathrm{F}} \\ &= \sum_{i=1}^{2d'} \min\left\{(\bar{\Sigma}_t)_{i,i}^2, (\bar{\Sigma}_t)_{d',d'}^2\right\} \left\|(\bar{V}_t)_{:,i}\right\|^2_{\widetilde{A}_t^{-1}} \le \frac{2d'}{\epsilon}(\bar{\Sigma}_t)_{d',d'}^2 \triangleq \Delta_t. \end{aligned}$$

The inequality uses the fact that $\widetilde{A}_t \succeq \epsilon I$ and $\left\|(\bar{V}_t)_{:,i}\right\|^2_{\widetilde{A}_t^{-1}} \le \left\|(\bar{V}_t)_{:,i}\right\|^2_{(\epsilon I)^{-1}} = \frac{1}{\epsilon}$. Substituting $(\bar{\Sigma}_t)_{i,i} = \sigma_i(S_t)$ completes the proof. ■

Lemma 16 *At the t-th round of Algorithm 6, in Algorithm 7, Δ_t also satisfies:*

$$\|\nabla g_t(\mathbf{y}_t)\|^2_{\widetilde{A}_t^{-2}} \le \left\langle \widetilde{A}_t^{-2}, \widetilde{A}_t - \widetilde{A}_{t-1} \right\rangle_{\mathrm{F}} + \frac{\Delta_t}{\epsilon}.$$

We omit the proof of Lemma 16, as it directly reuses the proof of Lemma 15.

Lemma 17 (Theorem 1.1 and Section 3 of Ghashami et al. [2016]) *In Algorithm 6, $\Delta_{1:T}$ satisfies that, for any $j \in [d']$,*

$$\Delta_{1:T} \triangleq \sum_{t=1}^{T} \Delta_t \leq \frac{2d'}{(d'-j+1)\epsilon} \sum_{i=j}^{d} \lambda_i \left(\sum_{t=1}^{T} \nabla g_t(\mathbf{y}_t) \nabla g_t(\mathbf{y}_t)^\mathsf{T} \right).$$

With the preceding lemmas characterizing the sketching error, we can prove Theorem 4 by bounding the difference between LightONS and LightONS.Sketch.

Proof of Theorem 4 We note that Lemma 11 holds for Algorithm 6, as the proof of Lemma 11 only requires A_t to be positive-definite. Thus by Lemmas 4 and 11 we obtain

$$\begin{aligned}
&f_t(\mathbf{x}_t) - f_t(\mathbf{u}) \overset{(9)}{\leq} \nabla g_t(\mathbf{y}_t)^\mathsf{T}(\mathbf{y}_t - \mathbf{u}) - \frac{\gamma_0}{2} \left(\nabla g_t(\mathbf{y}_t)^\mathsf{T}(\mathbf{y}_t - \mathbf{u}) \right)^2 \\
&\overset{(24)}{\leq} \frac{1}{2} \left(\frac{1}{\gamma_0} \|\nabla g_t(\mathbf{y}_t)\|^2_{\widetilde{A}_t^{-1}} + \gamma_0 \|\mathbf{y}_t - \mathbf{u}\|^2_{\widetilde{A}_t} - \gamma_0 \|\mathbf{y}_{t+1} - \mathbf{u}\|^2_{\widetilde{A}_t} \right) - \frac{\gamma_0}{2} \left(\nabla g_t(\mathbf{y}_t)^\mathsf{T}(\mathbf{y}_t - \mathbf{u}) \right)^2 \\
&\leq \frac{1}{2\gamma_0} \|\nabla g_t(\mathbf{y}_t)\|^2_{\widetilde{A}_t^{-1}} + \frac{\gamma_0}{2} \|\mathbf{y}_t - \mathbf{u}\|^2_{\widetilde{A}_{t-1}} - \frac{\gamma_0}{2} \|\mathbf{y}_{t+1} - \mathbf{u}\|^2_{\widetilde{A}_t},
\end{aligned}$$

where the last inequality uses the fact that $\widetilde{A}_t \preceq \widetilde{A}_{t-1} + \nabla g_t(\mathbf{y}_t) \nabla g_t(\mathbf{y}_t)^\mathsf{T}$, which is ensured by Algorithm 7. Then plugging Lemma 15 into the preceding inequality yields

$$\begin{aligned}
&f_t(\mathbf{x}_t) - f_t(\mathbf{u}) \\
&\leq \frac{1}{2\gamma_0} \left(\left\langle \widetilde{A}_t^{-1}, \widetilde{A}_t - \widetilde{A}_{t-1} \right\rangle_\mathrm{F} + \Delta_t \right) + \frac{\gamma_0}{2} \|\mathbf{y}_t - \mathbf{u}\|^2_{\widetilde{A}_{t-1}} - \frac{\gamma_0}{2} \|\mathbf{y}_{t+1} - \mathbf{u}\|^2_{\widetilde{A}_t} \\
&\leq \frac{1}{2\gamma_0} \left(\log\det(\widetilde{A}_t) - \log\det(\widetilde{A}_{t-1}) + \Delta_t \right) + \frac{\gamma_0}{2} \|\mathbf{y}_t - \mathbf{u}\|^2_{\widetilde{A}_{t-1}} - \frac{\gamma_0}{2} \|\mathbf{y}_{t+1} - \mathbf{u}\|^2_{\widetilde{A}_t},
\end{aligned}$$

where the inequality uses the fact that $\left\langle X^{-1}, X - Y \right\rangle_\mathrm{F} \leq \log\det(X) - \log\det(Y)$, which comes from the proof of Lemmas 5 and 6. Telescoping the preceding inequality yields

$$\sum_{t=1}^{T} (f_t(\mathbf{x}_t) - f_t(\mathbf{u})) \leq \frac{1}{2\gamma_0} \log \frac{\det(\widetilde{A}_T)}{\det(\widetilde{A}_0)} + \frac{1}{2\gamma_0} \Delta_{1:T} + \frac{\gamma_0}{2} \|\mathbf{y}_1 - \mathbf{u}\|^2_{\widetilde{A}_0},$$

where the logarithmic term is further bounded with Jensen's inequality, which differs slightly from Lemma 5 due to the number of non-zero eigenvalues:

$$\log \frac{\det(\widetilde{A}_T)}{\det(\widetilde{A}_0)} = \sum_{i=1}^{2d'} \log \left(1 + \frac{\sigma_i^2(S_T)}{\epsilon} \right) \leq 2d' \log \left(1 + \frac{\|S_T\|_\mathrm{F}^2}{2d'\epsilon} \right) \leq 2d' \log \left(1 + \frac{G^2}{2d'\epsilon} T \right).$$

Finally, the runtime follows from the following two parts:

- **Runtime aside from Algorithm 7.** Following the same analysis of Lemmas 2 and 14, overall runtime aside from Algorithm 7 is $O((k-1)^{-1}\sqrt{(d+\Delta_{1:T})T/\epsilon} \cdot d^\omega \log T)$ due to FastProj. (Algorithm 6 applies $k = 2$.) It suffices to verify the following inequality which follows from Lemma 16:

$$\Phi''_T \triangleq \sum_{t=1}^{T} \left\| \frac{1}{\gamma_0} \widetilde{A}_t^{-1} \nabla g_t(\mathbf{y}_t) \right\|_2^2 \leq \frac{1}{\gamma_0^2} \frac{d + \Delta_{1:T}}{\epsilon}.$$

- **Runtime of Algorithm 7.** Each time SVD is triggered, the last $d'+1$ rows of S_t become all-zero, thus SVD is triggered at most $\lceil T/(d'+1)\rceil$ times. The runtime of SVD is $O(d'^2 d \log T)$ [Golub and Van Loan, 2013] to achieve the desired accuracy that does not affect the regret bound. [5] The runtime of updating R_t is $O(d'd)$, as this can be implemented with two rank-one updates:

$$(A+\mathbf{u}\mathbf{v}^\mathsf{T})^{-1} = A^{-1} - \frac{1}{1+\mathbf{v}^\mathsf{T}A^{-1}\mathbf{u}} A^{-1}\mathbf{u}\mathbf{v}^\mathsf{T}A^{-1}.$$

Specifically,

$$\begin{aligned} R_t^{-1} &= \epsilon I + S_t S_t^\mathsf{T} = \epsilon I + \left(S_{t-1} + \mathbf{e}_{i_t}\nabla g_t(\mathbf{y}_t)^\mathsf{T}\right)\left(S_{t-1} + \mathbf{e}_{i_t}\nabla g_t(\mathbf{y}_t)^\mathsf{T}\right)^\mathsf{T} \\ &= \underbrace{\epsilon I + S_{t-1}S_{t-1}^\mathsf{T}}_{R_{t-1}^{-1}} + \mathbf{e}_{i_t}\underbrace{\left(S_{t-1}\nabla g_t(\mathbf{y}_t)\right)^\mathsf{T}}_{\mathbf{a}_t^\mathsf{T}} + \underbrace{\left(S_{t-1}\nabla g_t(\mathbf{y}_t) + \mathbf{e}_{i_t}\nabla g_t(\mathbf{y}_t)\nabla g_t(\mathbf{y}_t)^\mathsf{T}\right)}_{\mathbf{b}_t}\mathbf{e}_{i_t}^\mathsf{T}, \end{aligned}$$

where $\mathbf{e}_{i_t}$, $\mathbf{a}_t$ and $\mathbf{b}_t$ are $2d'$-dimensional vectors that can be computed in $O(d'd)$ time. Therefore, the overall runtime of Algorithm 7 is $O(d'^2 d(\log T)\lceil T/(d'+1)\rceil + d'dT) = O(d'dT\log T)$.

■

Comparison with OQNS. Mhammedi and Gatmiry [2023] mention the possibility of combining OQNS with sketching but provide neither algorithms nor analysis. Incorporating sketching further complicates OQNS's already intricate analysis, as the sketching error interacts with the log-barrier, Hessian approximation and decision updates in Eq. (4). In contrast, within the OMD framework, sketching errors naturally appear as an additive term in the regret bound via OMD's stability term, illustrated in Eq. (30), since OMD's regret decomposition is somewhat orthogonal to sketching.

5. The runtime of SVD can be improved to $O\big(d'^2(d+\log T)\big)$ and the factor $O(\log T)$ can be independent of the minimal singular value gap of S_t and only depends on the scale of singular values of S_t [Parlett, 1998].